



POLITIQUE DE SÉCURITÉ

Cours 2



Dr. Sarra Namane

Département d'informatique

Université Badji Mokhtar, Annaba

Promotion: Master 1 (RSI)

Année: 2023/2024

Introduction au modèle de contrôle d'accès

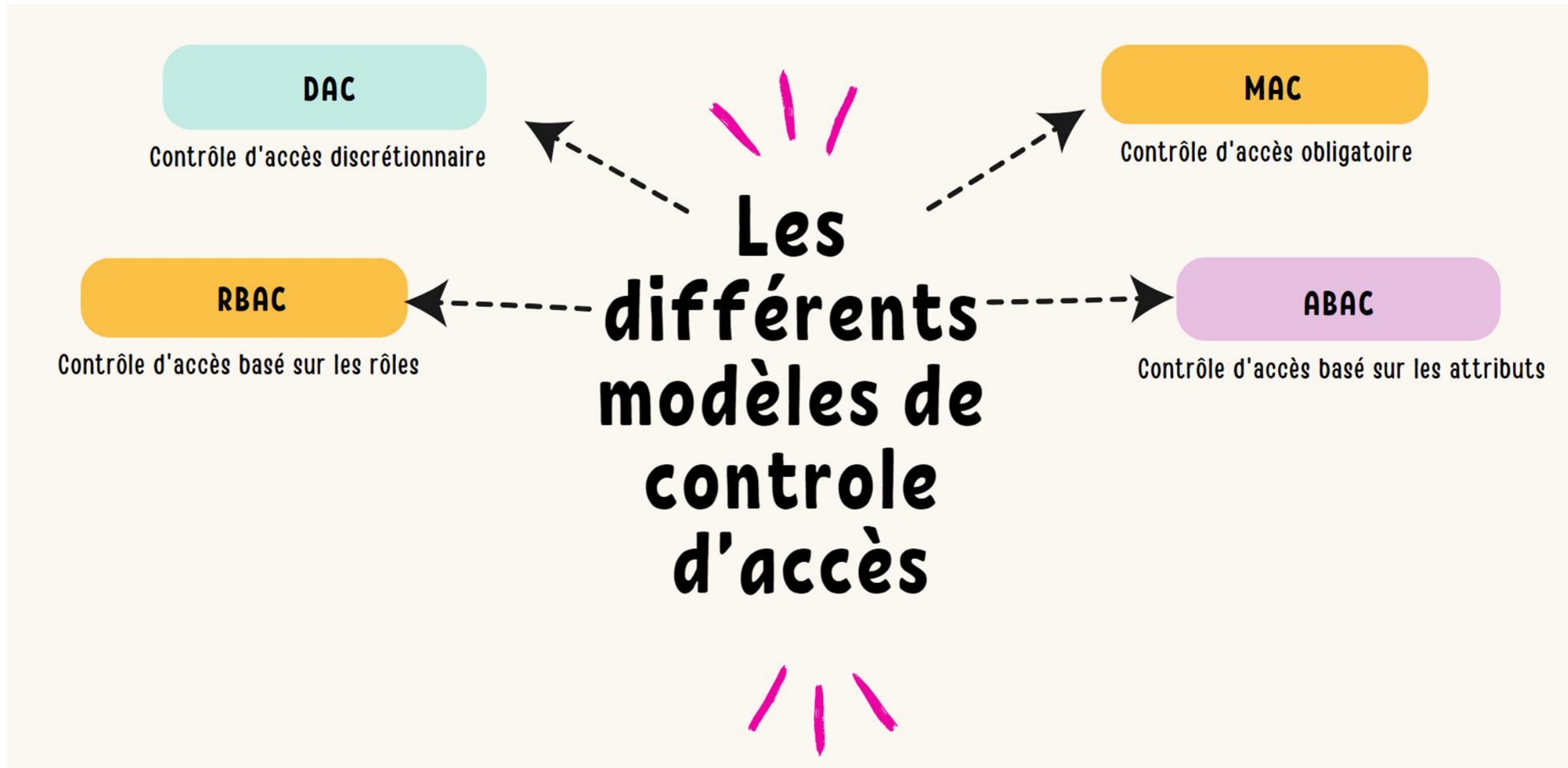
- Dans le domaine de l'informatique et des technologies de l'information, la sécurité joue un rôle crucial pour protéger les systèmes, les données et les utilisateurs contre les menaces potentielles.
- Les **modèles de sécurité** fournissent un cadre conceptuel essentiel pour comprendre, analyser et mettre en œuvre des mesures de sécurité efficaces.
- Les **modèles de sécurité** définissent les principes et les objectifs fondamentaux de la sécurité informatique, tels que la confidentialité, l'intégrité et la disponibilité des données. Ils fournissent un cadre conceptuel pour comprendre les menaces potentielles et élaborer des politiques de sécurité appropriées. **Les modèles de contrôle d'accès**, quant à eux, font partie intégrante des modèles de sécurité et se concentrent spécifiquement sur la gestion de l'accès aux ressources informatiques.



Définition d'un modèle de contrôle d'accès

- Un modèle de contrôle d'accès est un **cadre conceptuel formel** qui permet de décrire précisément le fonctionnement d'un système afin de le comprendre de manière approfondie.
- Il fournit une **représentation abstraite** du système, ce qui simplifie la compréhension de la politique de sécurité qui régit l'accès aux ressources.
- En utilisant ce modèle, les concepteurs peuvent spécifier les règles et les politiques de contrôle d'accès de manière claire et précise.
- De plus, le modèle facilite l'implémentation des mécanismes techniques nécessaires pour appliquer ces politiques et atteindre les objectifs de sécurité définis.
- En résumé, un modèle de contrôle d'accès permet de formaliser et de simplifier la gestion de l'accès aux ressources informatiques, tout en assurant la sécurité du système.

Les différents modèles de contrôle d'accès



Le modèle DAC

Définition 1

- La définition du contrôle d'accès discrétionnaire dans TCSEC (Trusted Computer System Evaluation Criteria) stipule que ce mécanisme permet de restreindre l'accès aux objets en fonction de l'identité des utilisateurs ou des groupes auxquels ils appartiennent. Ce contrôle est qualifié de "discrétionnaire" car il donne au sujet la capacité de transférer les autorisations d'accès à d'autres utilisateurs.
- En d'autres termes, le propriétaire de l'objet peut choisir de partager l'accès à cet objet avec d'autres utilisateurs, et cette décision est laissée à sa discrétion. Cela signifie que le sujet a le pouvoir discrétionnaire de déléguer les droits d'accès à d'autres utilisateurs selon ses propres critères et besoins.

Le modèle DAC (2)

Définition 2

- Dans la deuxième définition, le contrôle d'accès est qualifié de "discrétionnaire" car il repose sur l'identité du sujet, c'est-à-dire l'utilisateur ou le processus qui demande l'accès à un objet. Dans ce contexte, un "sujet" peut être un utilisateur individuel ou un groupe d'utilisateurs.
- La caractéristique principale du contrôle d'accès discrétionnaire est que les droits d'accès sont déterminés par le propriétaire ou le créateur de l'objet. Ce propriétaire a la discrétion de spécifier quels utilisateurs ou groupes d'utilisateurs ont accès à cet objet et quel type d'accès leur est accordé (lecture, écriture, exécution, etc.).
- De plus, dans un système de contrôle d'accès discrétionnaire, le sujet qui détient les droits d'accès à un objet a la faculté de transférer ces droits à d'autres utilisateurs, selon ses propres critères. Cela signifie que le propriétaire peut décider de partager l'accès à un objet avec d'autres utilisateurs en leur accordant des autorisations spécifiques.

Les différents modèles DAC

Modèle de contrôle d'accès discrétionnaire (DAC)

```
graph TD; A[Modèle de contrôle d'accès discrétionnaire (DAC)] --> B[Modèle de Lampson]; A --> C[Modèle Take-Grant]; A --> D[Modèle HRU];
```

Modèle de
Lampson

Modèle
Take-Grant

Modèle
HRU

Le modèle de Lampson

Le modèle de Lampson pour le contrôle d'accès, introduit en 1971, est basé sur la notion de droits d'accès spécifiés par une matrice de contrôle d'accès. Ce modèle repose sur trois composantes principales :

1. **Ensemble d'objets (O)** : Il s'agit des entités dans le système qui doivent être protégées. Ces objets peuvent être des fichiers, des répertoires, des dispositifs ou d'autres ressources informatiques.
2. **Ensemble de sujets (S)** : Ce sont les entités qui ont accès aux objets. Les sujets peuvent être des utilisateurs, des processus ou d'autres entités qui interagissent avec les objets du système.
3. **Matrice d'accès (M)** : C'est une structure de données qui spécifie les droits d'accès des sujets aux objets. Chaque entrée de la matrice représente une paire sujet-objet et indique les droits que le sujet possède sur l'objet correspondant. Par exemple, une entrée de la forme $M[s, o] = \{\text{lire, écrire}\}$ signifie que le sujet s a le droit de lire et d'écrire sur l'objet o .

En outre, l'ensemble R représente un ensemble de droits, tels que lire, écrire, exécuter, etc., qui peuvent être attribués aux sujets pour interagir avec les objets du système. Chaque droit r est défini comme une paire sujet-objet, notée $r \in [S, O]$, ce qui signifie qu'il est associé à une relation spécifique entre un sujet et un objet.

En résumé, le modèle de Lampson pour le contrôle d'accès organise les interactions entre les sujets et les objets à l'aide d'une matrice d'accès, spécifiant les droits d'accès des sujets aux objets, et cela est crucial pour assurer la sécurité et la protection des ressources du système.

Exemple d'une matrice de contrôle d'accès

E : écrire, L : lire

	Enregistrement médical	Enregistrement administratif	Prescriptions
Imene	E, L	E	L
Karim		E	L
Mohamed		L	
Nadine	E, L	L	E, L

Le modèle de Lampson (2)

Ces deux concepts, la liste de contrôle d'accès (ACL) et la liste de capacités (ou capability), sont deux approches différentes pour stocker et organiser les autorisations d'accès dans un système de contrôle d'accès.

1. Liste de contrôle d'accès (ACL) :

Dans ce modèle, la matrice d'accès est stockée par colonne, ce qui signifie que pour chaque objet, il existe une liste d'autorisations associée à chaque sujet. Chaque colonne de la matrice représente un objet et contient une liste des droits accordés à chaque sujet pour cet objet spécifique. Par exemple, pour un fichier donné, la colonne correspondante de la matrice ACL contiendrait une liste des utilisateurs autorisés et les actions qu'ils peuvent effectuer sur ce fichier.

2. Liste de capacités (ou capability) :

Dans ce modèle, la matrice d'accès est stockée par ligne, ce qui signifie que pour chaque sujet, il existe une liste d'autorisations associée à chaque objet. Chaque ligne de la matrice représente un sujet et contient une liste des droits que ce sujet possède sur chaque objet du système. Par exemple, pour un utilisateur donné, la ligne correspondante de la matrice de capacité contiendrait une liste des objets auxquels il peut accéder et les actions qu'il peut effectuer sur chacun d'eux.

En résumé, la différence principale entre l'ACL et la capacité réside dans la manière dont les autorisations sont stockées et organisées : l'ACL stocke les autorisations par objet, tandis que la capacité stocke les autorisations par sujet. Ces deux approches ont leurs avantages et inconvénients, et le choix entre les deux dépend des besoins spécifiques du système et des préférences de conception.

Exercice sur le modèle Lampson

Hypothèse :

Considérons trois utilisateurs : David, Emma et Félix.

Nous avons trois fichiers : rapport-financier, présentation_projet et feuille_calcul, appartenant respectivement à David, Emma et Félix.

De plus :

- David peut lire et écrire dans présentation_projet.
- Emma peut lire rapport_financier et écrire dans feuille_calcul.
- Félix peut lire et exécuter rapport_financier, lire et écrire dans présentation_projet et lire sur feuille_calcul.
- Chaque propriétaire peut exécuter les fichiers qui lui appartiennent.

Questions :

1. Représentez la matrice d'accès.
2. Effectuez les actions suivantes et représentez la nouvelle matrice d'accès :
 - Félix donne à Emma le droit de lire et d'écrire dans rapport_financier.
 - David retire le droit de lecture à Félix sur rapport _ financier.

Solution

	rapport_financier	présentation_projet	feuille_calcul
David	X	L,E	
Emma	L	X	E
Félix	L,X	L,E	L,X

L: Lire
E: Ecrire
X: Exécuter

Nouvelle matrice

	rapport_financier	présentation_projet	feuille_calcul
David	X	L,E	
Emma	L,E	X	E
Félix	X	L,E	L,X

Modèle de Harrison-Ruzzo-Ullman

Le modèle de Harrison-Ruzzo-Ullman (HRU) est une amélioration du modèle de contrôle d'accès discrétionnaire de Lampson. Dans ce modèle, une matrice de contrôle d'accès classique est utilisée, mais ce qui le distingue est que **HRU précise les commandes qui peuvent être appliquées à chaque objet.**

Contrairement au modèle de **Lampson**, où les autorisations sont simplement définies par des droits d'accès (lecture, écriture, etc.), le modèle HRU introduit une distinction entre les droits d'accès et les commandes. Les droits d'accès déterminent quelles actions un sujet peut effectuer sur un objet (par exemple, lire ou écrire), tandis que les commandes spécifient les opérations spécifiques qui peuvent être effectuées sur un objet (par exemple, copier, supprimer, renommer).

Ainsi, avec le modèle HRU, la matrice de contrôle d'accès spécifie non seulement les droits d'accès des sujets aux objets, mais elle précise également les commandes qui peuvent être appliquées à chaque objet. Cette distinction permet une granularité plus fine dans la gestion des autorisations et offre un meilleur contrôle sur les actions pouvant être effectuées dans le système.

Modèle de Harrison-Ruzzo-Ullman (2)

Les états de protection

Les états de protection sont un concept fondamental dans les modèles de contrôle d'accès. Voici une explication détaillée des composants et des aspects associés aux états de protection:

- **SC (Sujets)**: Il s'agit d'un ensemble dénombrable de constantes représentant les sujets du système. Chaque élément de cet ensemble est noté s ou s' .
- **OC (Objets)**: C'est également un ensemble dénombrable de constantes qui représente les objets du système. Chaque élément de cet ensemble est noté o ou o' .
- **R (Droits)**: Il s'agit d'un ensemble fini de droits, représentant les actions autorisées sur les objets du système. Chaque élément de cet ensemble est noté r ou r' .
- **Relation entre SC et OC**: Dans le contexte des modèles de contrôle d'accès, comme celui de Harrison, Ruzzo et Ullman, tous les sujets sont également considérés comme des objets. Cela est symbolisé par l'inclusion de l'ensemble SC dans l'ensemble OC, ce qui signifie que chaque sujet est également un objet.

Modèle de Harrison-Ruzzo-Ullman (3)

- **État de protection:** Il est représenté par un triplet (S, O, P) , où :
 - S est l'ensemble des sujets,
 - O est l'ensemble des objets,
 - P est une matrice d'accès qui spécifie les droits que chaque sujet a sur chaque objet. Pour chaque sujet s dans S et pour chaque objet o dans O , $P[s, o]$ définit l'ensemble des droits que le sujet s a sur l'objet o .
- Notation des états de protection : Les états de protection sont souvent représentés par les lettres Δ, Δ' .

Modèle de Harrison-Ruzzo-Ullman (4)

Les opérations primitives

Les **opérations primitives** sont des actions de base qui permettent de modifier l'état de protection dans un système de contrôle d'accès. Voici une explication détaillée des composants et des opérations primitives associées :

- **SV (Variables de sujets)**: Il s'agit d'un ensemble dénombrable de variables représentant les sujets du système. Chaque élément de cet ensemble est noté x ou x' .
- **OV (Variables d'objets)**: C'est également un ensemble dénombrable de variables représentant les objets du système. Chaque élément de cet ensemble est noté y ou y' .
- **Opérations primitives** : Il existe six opérations primitives qui permettent de modifier l'état de protection :
 1. **Créer le sujet x** : Cette opération crée un nouveau sujet dans le système, représenté par la variable x .

Modèle de Harrison-Ruzzo-Ullman (5)

- 2. Créer l'objet y :** Cette opération crée un nouvel objet dans le système, représenté par la variable y .
- 3. Introduire r dans $P[x, y]$:** Cette opération ajoute un droit r dans la matrice d'accès pour le sujet x sur l'objet y .
- 4. Détruire le sujet x :** Cette opération supprime le sujet x du système, ainsi que toutes ses associations avec les objets.
- 5. Détruire l'objet y :** Cette opération supprime l'objet y du système, ainsi que toutes les associations avec les sujets.
- 6. Supprimer r de $P[x, y]$:** Cette opération retire le droit r de la matrice d'accès pour le sujet x sur l'objet y .

- Notation des opérations primitives : Les opérations primitives sont souvent représentées par la lettre grecque π .

Modèle de Harrison-Ruzzo-Ullman (6)

Dans le modèle de Harrison-Ruzzo-Ullman (HRU), les opérations primitives sont appliquées indirectement à travers des commandes de la forme :

si C_1 et ... et C_i alors début π_1 ; ... ; π_j

Où $C_1, \dots, C_i$ sont des **conditions élémentaires** de la forme $r \in P[x, y]$ et $\pi_1, \dots, \pi_j$ sont des **opérations primitives**.

- **Conditions élémentaires:** Ce sont des conditions qui spécifient si un certain droit est présent dans la matrice d'accès pour une certaine paire sujet-objet. Par exemple, $r \in P[x, y]$ signifie que le droit r est présent pour le sujet x sur l'objet y .
- **Opérations primitives:** Ce sont les opérations de base qui modifient l'état de protection, comme expliqué précédemment.

Les commandes sont exécutées en remplaçant toutes les variables de la commande par des constantes. Si les conditions initiales $C_1, \dots, C_i$ sont évaluées à vrai dans l'état de protection courant, alors les opérations primitives $\pi_1, \dots, \pi_j$ sont exécutées séquentiellement.

Les commandes dans le modèle HRU sont souvent notées par la lettre grecque α .

Modèle de Harrison-Ruzzo-Ullman (7)

Assignment (θ) et Commande (α) : Dans ce contexte, l'assignation θ représente un ensemble de valeurs attribuées aux variables du système, tandis que la commande α est une instruction conditionnelle de la forme "si C_1 et ... et C_i alors début π_1 ; ... ; π_j fin". Les variables dans α sont remplacées par des constantes en utilisant Δ , ce qui signifie que la commande α est fixée pour une configuration donnée du système.

États de Protection (Δ et Δ') : Dans le modèle HRU, un état de protection est représenté par un triplet (S, O, P) , où S représente l'ensemble des sujets, O l'ensemble des objets et P les autorisations d'accès entre sujets et objets. Ainsi, Δ représente l'état de protection initial et Δ' l'état de protection résultant après l'application de la commande α .

Modèle de Harrison-Ruzzo-Ullman (8)

Dérivation des États de Protection

On dit que l'état de protection Δ' est dérivable de Δ en utilisant θ et α si l'une des deux conditions suivantes est satisfaite :

a. Si les conditions de la commande α ne sont pas satisfaites, alors l'état de protection reste inchangé ($\Delta' = \Delta$). Cela signifie que l'exécution de la commande α n'a pas d'impact sur l'état de protection.

b. Sinon, il existe une séquence d'états de protection $\Delta_0, \Delta_1, \dots, \Delta_j$, où Δ_0 correspond à l'état initial Δ et Δ_j est l'état final Δ' , tels que chaque état Δ_i est obtenu en appliquant les actions π_i de la commande α à l'état précédent Δ_{i-1} . Cela signifie que l'exécution de la commande α entraîne une série de changements dans l'état de protection, conduisant finalement à l'état de protection Δ' .

$$\Delta = \Delta_0 \rightarrow \Delta_1 \rightarrow \Delta_2 \rightarrow \Delta_3 \rightarrow \Delta_4 \rightarrow \dots \rightarrow \Delta_j = \Delta'$$

Modèle de Harrison-Ruzzo-Ullman (9)

Exemple sur Dérivation des États de Protection

Supposons que nous ayons un système informatique avec trois sujets (utilisateurs) : Alice, Bob et Carol, et trois objets (ressources) : Fichier A, Fichier B et Fichier C. Les autorisations d'accès sont les suivantes :

- Alice a accès en lecture au Fichier A.
- Bob a accès en lecture et écriture au Fichier B.
- Carol a accès en lecture au Fichier C.

L'état de protection initial ($\triangle$) serait représenté comme suit :

$\triangle = (S, O, P)$ $S = \{\text{Alice, Bob, Carol}\}$

$O = \{\text{Fichier A, Fichier B, Fichier C}\}$

$P = \{ (\text{Alice, Fichier A, lecture}), (\text{Bob, Fichier B, lecture}), (\text{Bob, Fichier B, écriture}), (\text{Carol, Fichier C, lecture}) \}$

Maintenant, supposons que la commande α soit "**si Alice a accès en lecture au Fichier A, alors supprimer l'accès en lecture de Carol au Fichier C**".

Dans ce cas, la condition de la commande α est **satisfaite** car Alice a effectivement accès en lecture au Fichier A. Ainsi, nous devons modifier l'état de protection pour refléter cela. Cependant, comme Bob n'a pas accès au Fichier C, la deuxième action de la commande α ne sera pas appliquée.

La séquence des états de protection serait la suivante :

1. $\triangle_0 = (S, O, P)$: état initial

2. $\triangle_1$: Alice supprime l'accès en lecture de Carol au Fichier C

- $\triangle_1 = (S, O, P')$ où $\triangle' = \{ (\text{Alice, Fichier A, lecture}), (\text{Bob, Fichier B, lecture}), (\text{Bob, Fichier B, écriture}) \}$

Ainsi, dans cet exemple, l'état de protection $\triangle'$ est dérivable de $\triangle$ en utilisant l'assignation et la commande spécifiées, car les actions correspondantes de la commande α sont appliquées avec succès, entraînant un changement dans l'état de protection initial.

Modèle de Harrison-Ruzzo-Ullman (10)

Transition des états

La transition des états dans le modèle HRU est définie par les actions suivantes :

1. **"Créer le sujet x"**: Si x n'est pas déjà un sujet ($\theta(x)$ n'appartient pas à S), alors l'état transitionnel est défini comme suit :

1. S' est l'ensemble des sujets après la transition, obtenu en ajoutant $\theta(x)$ à l'ensemble initial S.
2. O' est l'ensemble des objets après la transition, obtenu en ajoutant $\theta(x)$ à l'ensemble initial O.
3. La matrice P' conserve les mêmes valeurs que P pour tous les états et actions (s, o), sauf pour $(\theta(x), o)$ où $P'[\theta(x), o]$ est vide pour tout o dans O' et pour (s, $\theta(x)$) où $P'[s, \theta(x)]$ est vide pour tout s dans S'.

$(x) \notin S, S' = S \cup \{\theta(x)\}, O' = O \cup \{\theta(x)\}, P'[s, o] = P[s, o] \text{ pour tout } (s, o) \in S \times O, P'[\theta(x), o] = \emptyset;$

2. **"Détruire le sujet x"**: Si $\theta(x)$ est un sujet, alors l'état transitionnel est défini comme suit :

1. S' est l'ensemble des sujets après la transition, obtenu en supprimant $\theta(x)$ de l'ensemble initial S.
2. O' est l'ensemble des objets après la transition, obtenu en supprimant $\theta(x)$ de l'ensemble initial O.
3. La matrice P' conserve les mêmes valeurs que P pour tous les états et actions (s, o) dans $S' \times O'$.

$\theta(x) \in S, S' = S \setminus \{\theta(x)\}, O' = O \setminus \{\theta(x)\} \text{ et } P'[s, o] = P[s, o] \text{ pour tout } (s, o) \in S' \times O'.$

3. **"Créer l'objet y"**: Si y n'est pas déjà un objet ($\theta(y)$ n'appartient pas à O), alors l'état transitionnel est défini comme suit :

1. S' est conservé tel quel.
2. O' est l'ensemble des objets après la transition, obtenu en ajoutant $\theta(y)$ à l'ensemble initial O.
3. La matrice P' conserve les mêmes valeurs que P pour tous les états et actions (s, o) dans $S \times O$, sauf pour (s, $\theta(y)$) où $P'[s, \theta(y)]$ est vide pour tout s dans S'.

$\theta(y) \notin O, S' = S, O' = O \cup \{\theta(y)\}, P'[s, o] = P[s, o] \text{ pour tout } (s, o) \in S \times O \text{ et } P'[s, \theta(y)] = \emptyset; \text{ pour tout } s \in S'.$

Modèle de Harrison-Ruzzo-Ullman (11)

Transition des états (2)

4. "**Détruire l'objet y** ": Si $\theta(y)$ est un objet qui n'est pas associé à un sujet, alors l'état transitionnel est défini comme suit :

1. S' est conservé tel quel.
2. O' est l'ensemble des objets après la transition, obtenu en supprimant $\theta(y)$ de l'ensemble initial O .
3. La matrice P' conserve les mêmes valeurs que P pour tous les états et actions (s, o) dans $S' \times O'$.

$\theta(y) \in O \setminus S, S' = S, O' = O \setminus \{\theta(y)\}$ et $P'[s, o] = P[s, o]$ pour tout $(s, o) \in S' \times O'$.

5. "**Introduire r dans $P(x, y)$** ": Si $\theta(x)$ est un sujet et $\theta(y)$ est un objet, alors l'état transitionnel est défini comme suit :

1. S' et O' sont conservés tels qu'ils sont.
2. La matrice P' conserve les mêmes valeurs que P pour tous les états et actions (s, o) dans $S \times O$, sauf pour $(\theta(x), \theta(y))$ où r est ajouté à la liste des permissions.

$\theta(x) \in S, \theta(y) \in O, S' = S, O' = O$ et pour tout $(s, o) \in S \times O$: $P'[s, o] = \begin{cases} P[s, o] \cup \{r\} & \text{si } s = \theta(x) \text{ et } o = \theta(y) \\ P[s, o] & \text{sinon} \end{cases}$

6. "**Supprimer r de $P(x, y)$** ": Si $\theta(x)$ est un sujet et $\theta(y)$ est un objet, alors l'état transitionnel est défini comme suit

1. S' et O' sont conservés tels qu'ils sont.
2. La matrice P' conserve les mêmes valeurs que P pour tous les états et actions (s, o) dans $S \times O$, sauf pour $(\theta(x), \theta(y))$ où r est retiré de la liste des permissions.

$\theta(x) \in S, \theta(y) \in O, S' = S, O' = O$ et pour tout $(s, o) \in S \times O$: $P'[s, o] = \begin{cases} P[s, o] \setminus \{r\} & \text{si } s = \theta(x) \text{ et } o = \theta(y) \\ P[s, o] & \text{sinon} \end{cases}$

Modèle de Harrison-Ruzzo-Ullman (12)

Exemple de commande

- **Création de fichier :**

Command créer-fichier(p,f)

créer_objet(f)

entrer(own,M[p,f])

entrer (r,M[p,f])

entrer (w,M[p,f])

Fin

- **Donner un accès en lecture sur un fichier :**

Command grand-read(p,f,q)

Si

own $\in$ M[p,f]

Alors

entrer(r,M[q,f])

Fin

**Commande qui indique
la c création d'un fichier
f par le processus P**

**Commande qui permet à un
sujet p propriétaire du fichier
f d'accorder un accès en
lecture à un autre sujet
q sur ce fichier**

Modèle de Harrison-Ruzzo-Ullman (13)

Exercice

Objets utilisateurs	Fichier1	Fichier 2	Fichier 3	Fichier 4
Ramzi	owns	read	read	
Riad		owns	read, write	owns
Billel	read	read ,write	owns	Read, Write
Nedjma			Read	Read, write

- Définir une commande confer.read qui permet à l'utilisateur Ramzi de donner un accès en lecture sur un fichier 1 à l'utilisateur Riad.

Modèle de Harrison-Ruzzo-Ullman (14)

L'automate d'accessibilité global

si le nombre de sujets et objets créés reste fini, il est possible d'explorer toutes les évolutions possibles du système .

- Cela signifie que dans un système où il y a un nombre limité de sujets (utilisateurs) et d'objets (ressources), il est théoriquement possible d'analyser toutes les combinaisons d'actions possibles que ces sujets peuvent effectuer sur ces objets. Cela permet de déterminer l'état du système à tout moment et de prévoir toutes les évolutions possibles.
- **L'automate d'accessibilité global** est un moyen de représenter toutes les combinaisons d'actions possibles dans un système. Dans cet exemple, l'automate d'accessibilité global montrerait tous les états possibles du système après l'exécution de chaque action. Cela permettrait de visualiser toutes les interactions entre les sujets et les objets et de déterminer les autorisations accordées ou révoquées à chaque étape. Cela aiderait également à identifier les éventuelles incohérences ou violations de sécurité dans le système.

Modèle de Harrison-Ruzzo-Ullman (15)

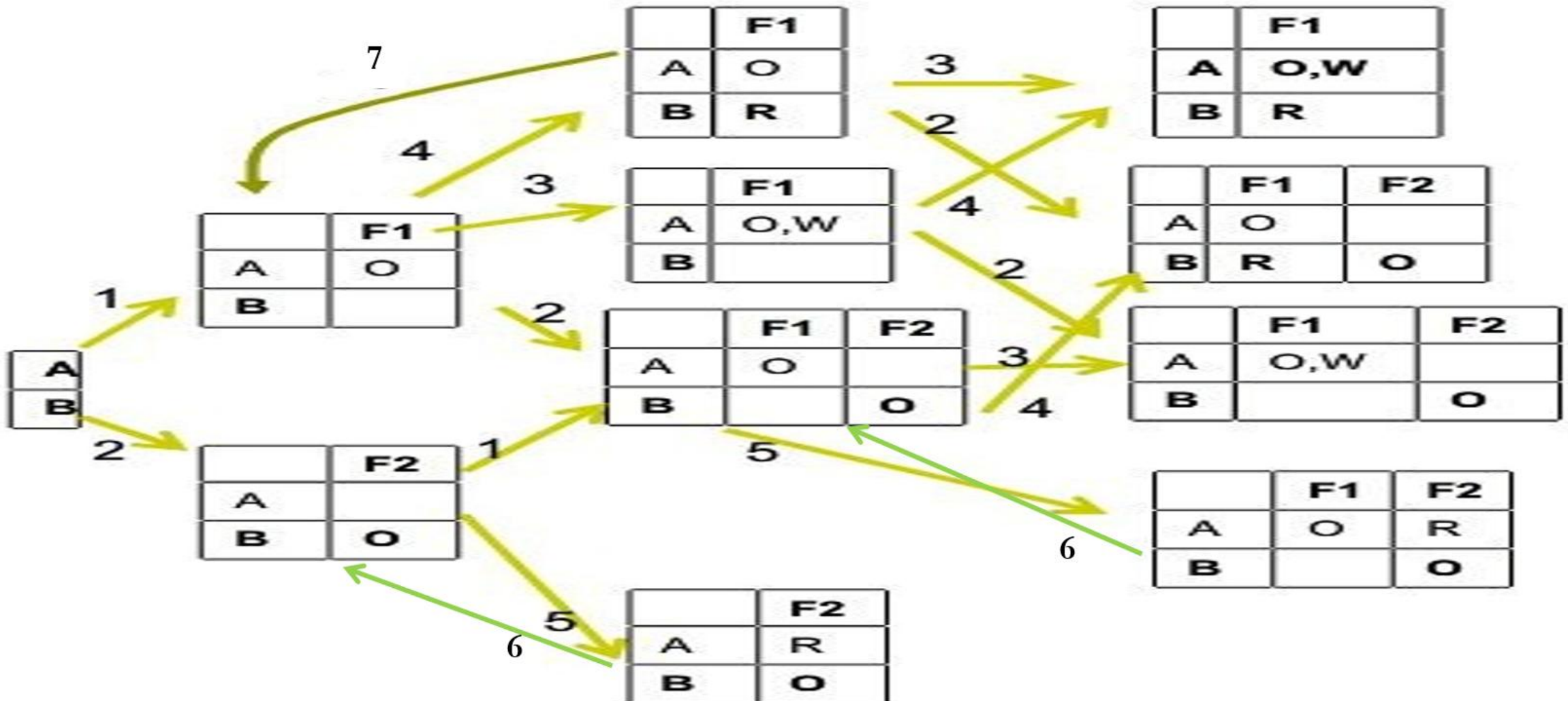
Exemple

Dans l'exemple donné, le système est initialement constitué de deux sujets, A et B, et aucun objet n'est encore créé. Les étapes suivantes décrivent les actions effectuées par ces sujets sur les objets :

1. A crée un objet F1
2. B crée un objet F2
3. A autorise A à écrire dans F1
4. A autorise B à lire dans F1
5. B autorise A à lire F2
6. Si A a le droit de lire F2, B lui retire ce droit
7. Si B a le droit de lire F1, A lui retire ce droit

Modèle de Harrison-Ruzzo-Ullman (16)

L'automate d'accessibilité global de l'exemple



Modèle Take-Grant

Le modèle Take-Grant, introduit par Jones et al. en 1976, est une variante initiale du modèle HRU (Harrison, Ruzzo, Ullman). Il se présente sous la forme d'un graphe dont les nœuds représentent des sujets ou des objets, et les arcs représentent les droits d'accès entre ces entités. Ce modèle offre une alternative à la matrice d'accès pour représenter les autorisations dans un système.

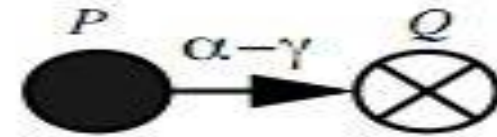
Les principales caractéristiques du modèle Take-Grant sont les suivantes :

1. **Structure du graphe:** Le graphe représente les autorisations d'accès entre les sujets et les objets. Chaque nœud représente un sujet ou un objet, et les arcs représentent les droits d'accès entre eux.
2. **Commandes restreintes:** Les commandes sont réparties en quatre catégories distinctes, ce qui restreint les actions possibles dans le système :
 - **La commande "create"** permet de créer un objet et d'attribuer initialement un droit d'accès à un sujet sur cet objet.
 - **La commande "remove"** permet de retirer un droit d'accès d'un sujet sur un objet.
 - **Les commandes de type "take"** permettent à un sujet possédant le droit spécial "t" sur un autre sujet d'obtenir les droits d'accès que ce sujet possède sur les objets.
 - **Les commandes de type "grant"** permettent à un sujet possédant déjà un droit d'accès sur un objet ainsi que le droit spécial "g" sur un autre sujet de céder ce droit d'accès à ce dernier sujet.

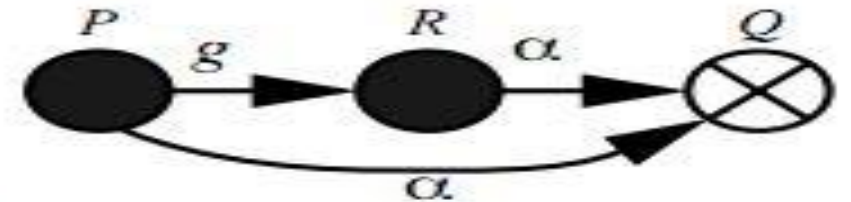
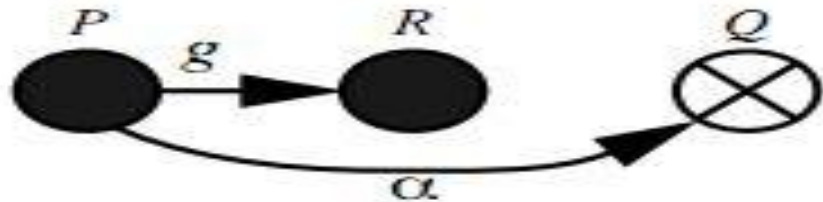
Modèle Take-Grant (2)



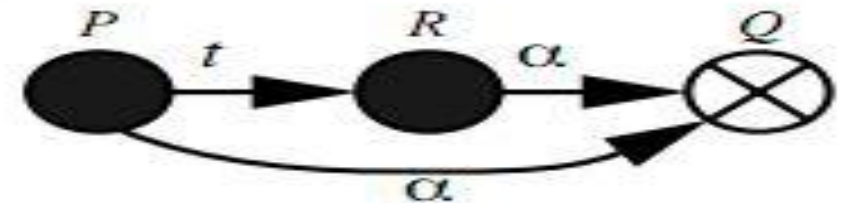
(a) Règle create



(b) Règle remove



(c) Règle grant



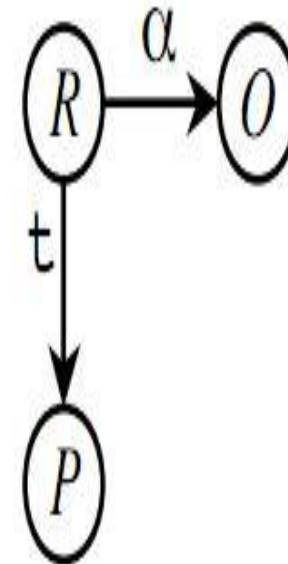
(d) Règle take

Modèle Take-Grant (3)

Les règles de contrôle d'accès peuvent sembler simples individuellement, mais leur combinaison peut conduire le système à des états d'insécurité. En effet, l'application successive de plusieurs règles, lorsqu'elles sont judicieusement choisies, peut entraîner l'octroi de nouveaux droits à des sujets, ce qui peut compromettre certains objectifs de sécurité.

Prenons l'exemple d'un graphe de protection illustré dans la figure suivante, extrait de [Dacier 1994]. Ce graphe contient deux sujets, P et R, ainsi qu'un objet O. Dans l'état de protection initial, le sujet R possède le droit α sur l'objet O et le droit t sur le sujet P.

Considérons un objectif de sécurité stipulant que le système est considéré comme non sécurisé si le sujet P parvient à acquérir le droit α sur l'objet O.



Le modèle MAC

Le contrôle d'accès obligatoire (Mandatory Access Control, MAC) repose sur le principe selon lequel la politique de sécurité appliquée à un système ne peut pas être modifiée par les utilisateurs du système eux-mêmes. Contrairement aux modèles de contrôle d'accès discrétionnaire, où les utilisateurs ont un certain contrôle sur les autorisations d'accès, dans un environnement MAC, l'attribution des permissions est gérée par une entité tierce et les utilisateurs ne peuvent en aucun cas intervenir dans ce processus.

Dans le modèle MAC, les politiques de sécurité imposent que les décisions de protection ne soient pas prises par les propriétaires des objets concernés, mais soient plutôt imposées par le système lui-même. Cela signifie que les règles de sécurité sont incontournables et garantissent l'atteinte des objectifs de sécurité visés.

Dans un système MAC, l'accès aux objets est basé sur le niveau de sensibilité de l'information contenue dans ces objets. L'autorisation d'accéder à un objet est accordée à un sujet si le niveau d'autorisation de ce sujet est en accord avec le niveau de sensibilité de l'information de l'objet.

Par exemple, si un sujet a un niveau d'autorisation qui correspond au niveau de sensibilité requis pour accéder à un certain objet, alors ce sujet sera autorisé à accéder à cet objet. Sinon, l'accès sera refusé.

Le modèle MAC (2)

La sécurité multi-niveaux

La sécurité multi-niveaux est une approche utilisée dans les systèmes informatiques multi-utilisateurs pour gérer les informations en fonction de leur sensibilité. Dans ce modèle, les informations sont classées en différents niveaux de sensibilité, et des mécanismes sont mis en place pour garantir que seuls les utilisateurs autorisés ont accès à des informations sensibles.

Voici quelques points clés à retenir sur la sécurité multi-niveaux :

1. Classification des informations sensibles: Les informations sont classées en fonction de leur importance et de leur sensibilité. Par exemple, certaines informations peuvent être classées comme "Non classifiées", tandis que d'autres peuvent être classées comme "Confidentielles", "Secretes" ou "Top Secret", en fonction du niveau de confidentialité requis.

2. Niveaux de sécurité pour les sujets et les objets: Dans ce modèle, chaque sujet (utilisateur ou processus) et chaque objet (ressource ou fichier) se voient attribuer un niveau de sécurité spécifique. Le niveau de sécurité d'un sujet est souvent appelé "Habilitation", tandis que le niveau de sécurité d'un objet est appelé "Classification". Ces niveaux de sécurité déterminent qui peut accéder à quelles informations.

Le modèle MAC (3)

La sécurité multi-niveaux (2)

3. Contrôle d'accès basé sur les niveaux de sécurité: Les mécanismes de contrôle d'accès sont mis en place pour garantir que seuls les sujets autorisés peuvent accéder à des objets ayant un niveau de classification égal ou inférieur à leur niveau d'habilitation. Par exemple, un utilisateur avec une habilitation "Secret" peut accéder à des informations classées "Confidentielles" mais pas à des informations classées "Top Secret".

4. Protection contre la divulgation non autorisée: L'objectif principal de la sécurité multi-niveaux est de prévenir la divulgation non autorisée d'informations sensibles. En limitant l'accès aux informations en fonction des niveaux de sécurité, le système garantit que seules les personnes autorisées peuvent accéder aux données appropriées, réduisant ainsi les risques de fuites ou de compromissions.

Le modèle MAC (4)

Dans le modèle de contrôle d'accès décrit, les concepts sont présentés à l'aide des notations suivantes :

1. **L**: C'est un ensemble de niveaux de sécurité, représentant les différents niveaux de sensibilité des informations. Ces niveaux sont ordonnés selon une relation d'ordre partiel $\leq$, ce qui signifie qu'un niveau de sécurité peut être inférieur, égal ou supérieur à un autre niveau de sécurité.

2. **fS: S $\rightarrow$ L**: Cette fonction assigne à chaque sujet (utilisateur ou processus) son niveau d'habilitation, c'est-à-dire le niveau de sécurité auquel il est autorisé à accéder. Par exemple, un sujet peut être autorisé à un niveau de sécurité "Confidentiel" ou "Secret".

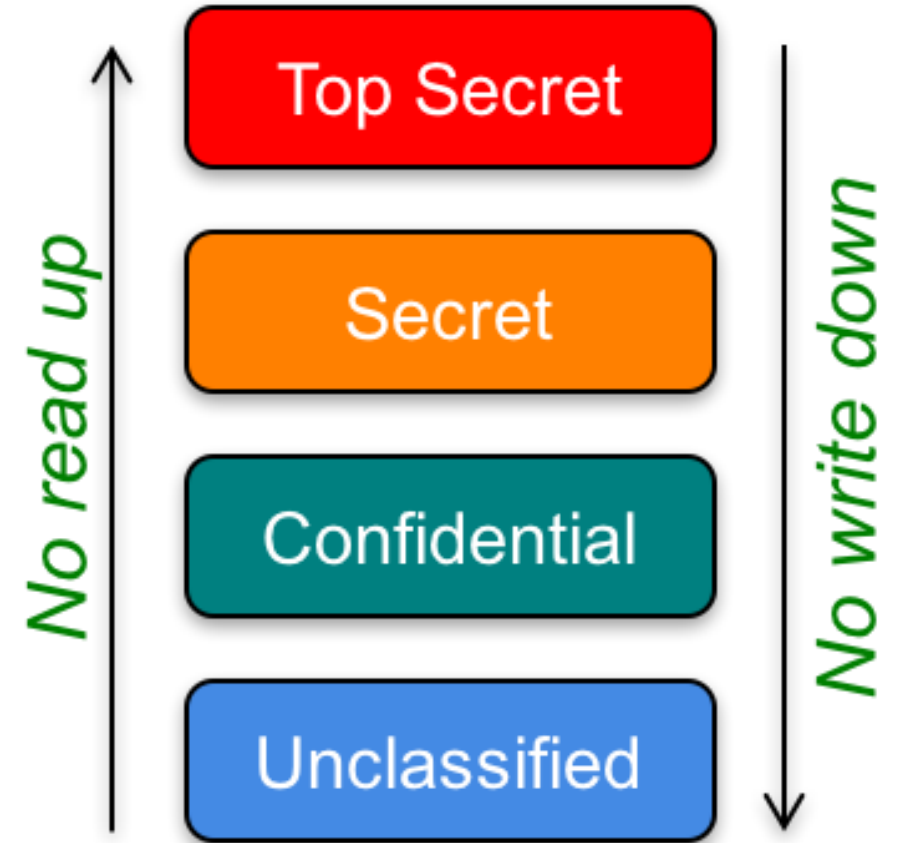
3. **fO: O $\rightarrow$ L**: Cette fonction attribue à chaque objet (ressource ou fichier) sa classification, c'est-à-dire le niveau de sécurité de l'objet lui-même. Par exemple, un fichier peut être classé comme "Confidentiel" ou "Secret".

4. **Mso**: C'est la matrice de contrôle d'accès qui associe à chaque couple (sujet, objet) un ensemble de droits d'accès. Cette matrice détermine les autorisations accordées à chaque sujet pour accéder à chaque objet. Par exemple, un sujet peut avoir le droit de lire ou d'écrire dans un certain objet en fonction de son niveau d'habilitation et de la classification de l'objet.

Le modèle MAC (5)

Dans ce modèle, les règles de contrôle d'accès suivantes sont établies :

- Un sujet est autorisé à lire des informations si son niveau d'habilitation est supérieur à la classification de ces informations. Cela signifie qu'un sujet ne peut lire des informations que s'il a un niveau de sécurité égal ou supérieur à celui des informations.
- Un sujet est autorisé à écrire dans des objets si son niveau d'habilitation est inférieur à la classification de ces objets. Cela signifie qu'un sujet ne peut écrire dans un objet que si son niveau de sécurité est inférieur ou égal à celui de l'objet.



Le modèle de Bell et La Padula

Le modèle de Bell et LaPadula, développé par David Elliott Bell et Len LaPadula pour le département de la défense américaine, formalise la sécurité multi-niveaux et met l'accent sur la confidentialité des données dans divers systèmes informatiques tels que les systèmes d'exploitation et les systèmes de gestion de bases de données.

À la différence du modèle proposé par Lampson, le modèle de Bell et LaPadula se concentre sur les flux d'information plutôt que sur l'accès direct aux ressources. Il est basé sur les concepts de sujets (utilisateurs ou processus), d'objets (ressources ou données) et de modes d'accès, mais il exprime les objectifs de sécurité en termes de circulation et de contrôle des informations plutôt qu'en termes d'accès.

Dans ce modèle, les droits d'accès dépendent des classifications attribuées aux objets et des habilitations attribuées aux sujets. Les classifications représentent les niveaux de sensibilité des informations et les habilitations représentent les niveaux de confiance ou d'autorisation des sujets à quelles ressources.

Le modèle de Bell et La Padula (2)

Les règles obligatoires imposées par le modèle de Bell et LaPadula visent à prévenir la circulation des informations d'un niveau de sécurité élevé vers un niveau de sécurité plus bas. Ces règles assurent que les informations classifiées à un niveau de sécurité élevé ne peuvent être transférées vers des sujets ou des objets classifiés à un niveau de sécurité inférieur, préservant ainsi la confidentialité des données.

De plus, le modèle introduit la notion de niveaux de sécurité formant un ensemble partiellement ordonné ($L; \leq$), où les niveaux de sécurité sont organisés hiérarchiquement pour déterminer les autorisations d'accès en fonction de la sensibilité des informations. Cela permet de définir clairement les restrictions et les autorisations d'accès pour chaque niveau de sécurité, contribuant ainsi à renforcer la sécurité et la confidentialité des systèmes informatiques.

Dans le modèle de Bell et LaPadula, la sécurité est abordée sous l'angle de la circulation et du contrôle des informations plutôt que de l'accès direct aux ressources. Cela signifie que le modèle se concentre sur la manière dont les informations se déplacent à travers le système et sur la manière dont elles sont contrôlées pour garantir la confidentialité et l'intégrité des données, plutôt que sur qui peut accéder.

Le modèle de Bell et La Padula (3)

Dans le modèle de contrôle d'accès de Bell-LaPadula, chaque sujet et chaque objet du système d'information se voient attribuer un niveau de sécurité. Ce niveau de sécurité est crucial pour déterminer les autorisations d'accès des sujets aux objets. Voici quelques points clés pour comprendre ce modèle :

1. Niveau d'habilitation et niveau de classification: Chaque sujet (utilisateur ou processus) se voit attribuer un niveau d'habilitation, tandis que chaque objet (donnée ou ressource) se voit attribuer un niveau de classification. Ces niveaux sont utilisés pour évaluer les autorisations d'accès.

2. Fonctions d'habilitation et de classification: Les fonctions f_s et f_o assignent respectivement les niveaux d'habilitation et de classification aux sujets et aux objets. Ces fonctions déterminent les niveaux de sécurité associés à chaque sujet et à chaque objet dans le système.

La fonction d'habilitation est notée $f_s: S \rightarrow L$ et celle de classification $f_o: O \rightarrow L$.

Le modèle de Bell et La Padula (4)

3. **Propriété de tranquillité forte:** Une caractéristique importante du modèle est la propriété de tranquillité forte, qui stipule que les niveaux de sécurité attribués aux sujets et aux objets ne peuvent pas être modifiés une fois qu'ils ont été établis. Cela garantit la stabilité et la prévisibilité du système de contrôle d'accès.

4. **Propriété de sécurité simple :** La propriété de sécurité simple, également connue sous le nom de principe "no read up", est une règle fondamentale du modèle Bell-LaPadula. Elle stipule qu'un sujet ne peut accéder à un objet que si le niveau d'habilitation du sujet est supérieur ou égal au niveau de classification de l'objet. En d'autres termes, un sujet ne peut pas lire des informations classifiées à un niveau de sécurité plus élevé que le sien. Cette règle vise à prévenir la divulgation non autorisée d'informations sensibles et à garantir la confidentialité des données.

Formellement : $\forall s \in S; \forall o \in O; (s; o; \text{read}) \in M \longrightarrow f_O(o) \leq f_S(s)$

Top secret

O

S

Secret

O

S

Confidentiel

S

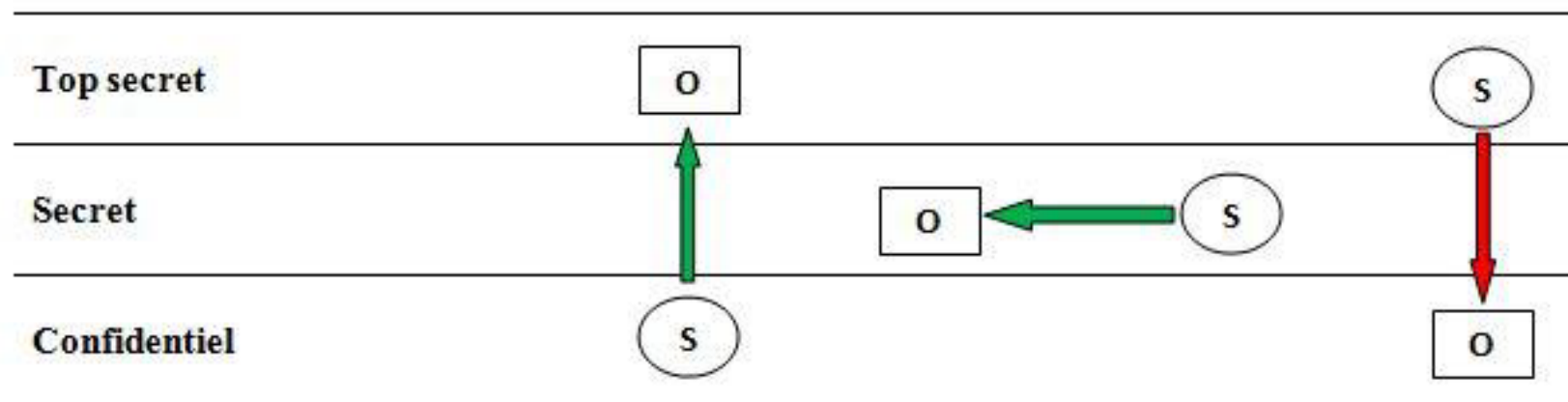
O



Le modèle de Bell et La Padula (5)

5. **La propriété étoile:** La propriété étoile du modèle de sécurité Bell-LaPadula établit une règle fondamentale dans la gestion de la confidentialité des informations : un sujet ne peut écrire dans un objet que si son niveau d'accréditation est inférieur ou égal au niveau de classification de cet objet. Cette règle est résumée par l'expression "ne pas écrire en bas". En d'autres termes, elle garantit que les utilisateurs ne peuvent pas accéder à des informations plus sensibles que leur niveau d'autorisation ne le permet, évitant ainsi les fuites de données et préservant l'intégrité du système.

Formellement : $\forall s \in S; \forall o \in O; (s; o; \text{write}) \in M \Rightarrow f_S(s) \leq f_O(o)$.



Variantes du modèle de Bell et La Padula

Dans le domaine de la sécurité informatique, des variantes du modèle Bell-LaPadula ont été introduites pour offrir plus de souplesse et d'expressivité. Ces variantes traduisent les notions d'observation et de modification de l'information non seulement par les opérations élémentaires de lecture et d'écriture, mais également par d'autres actions telles que l'exécution et l'ajout.

L'action "**Exécuter**" permet un accès à l'information sans la modifier ni l'observer, par exemple dans le cas où un programme est simplement exécuté sans lire ni écrire de données.

L'action "**Lire**" permet d'observer l'information sans la modifier, ce qui peut être utile dans des scénarios où la simple consultation des données est nécessaire.

L'action "**Ajouter**" (ou "append") permet de modifier l'information sans l'observer, par exemple lors de l'ajout de données dans des fichiers de journalisation d'audit ou lors de l'ajout de chemins dans des fichiers de configuration lors de l'installation de logiciels.

Enfin, l'action "**Écrire**" englobe à la fois l'observation et la modification de l'information, permettant ainsi un accès complet aux données.

Ces différentes actions offrent une granularité plus fine dans la gestion des autorisations d'accès aux informations, permettant ainsi de mieux répondre aux besoins spécifiques des applications et des environnements de sécurité.

Le modèle de sécurité peut être représenté par une machine à états où chaque état est défini par un triplet (S, O, M) , où :

S : représente un ensemble de sujets, qui sont les entités (utilisateurs, processus, etc.) qui accèdent aux objets.

O : représente un ensemble d'objets, qui sont les ressources (fichiers, bases de données, etc.) auxquelles les sujets accèdent.

A : représente un ensemble d'opérations d'accès, qui inclut des actions telles que exécuter, lire, ajouter et écrire.

un ensemble de niveaux de sécurité muni d'une relation d'ordre partiel, qui définit la hiérarchie des niveaux de sécurité.

B : représente l'ensemble de tous les états possibles de la machine à états.

M : représente l'ensemble des matrices (M_{so}) où $s \in S$ et $o \in O$. Une matrice M_{so} spécifie les autorisations d'accès des sujets aux objets, indiquant quelles opérations chaque sujet est autorisé à effectuer sur chaque objet.

$F \subset L_s^* \times L_o^* \times L_o$: représente l'ensemble des niveaux de sécurité dans un état donné. f_s est le niveau de sécurité maximal qu'un sujet peut avoir, f_c est le niveau de sécurité actuel de chaque sujet et f_o est la classification de l'objet.

Les deux règles de contrôle d'accès dans le contexte de la sécurité informatique sont les suivantes :

La propriété simple : Cette règle stipule qu'un état est considéré comme sécurisé si, pour chaque sujet s qui observe un objet o , le niveau de sécurité maximal du sujet ($fs(s)$) domine le niveau de sécurité de l'objet ($fo(o)$). Formellement, un état satisfait la propriété simple si pour chaque triplet (s, o, a) dans l'ensemble des états possibles (b), on a $fo(o) \leq fs(s)$. En d'autres termes, un sujet ne peut observer un objet que si son niveau de sécurité est au moins aussi élevé que celui de l'objet.

$$\forall (s, o, a) \in b, \Rightarrow fo(o) \leq fs(s)$$

La propriété étoile : Cette règle indique qu'un état est considéré comme sécurisé si, pour chaque sujet s qui modifie un objet o , le niveau de sécurité de l'objet ($fo(o)$) domine le niveau courant de sécurité du sujet ($fc(s)$). Formellement, un état est considéré comme sécurisé si pour chaque triplet (s, o, a) dans l'ensemble des états possibles (b), on a $fc(s) \leq fo(o)$. En d'autres termes, un sujet ne peut modifier un objet que si le niveau de sécurité de l'objet est au moins aussi élevé que le niveau de sécurité actuel du sujet.

$$\forall (s, o, a) \in b, \Rightarrow fc(s) \leq fo(o)$$

En résumé, ces deux règles de contrôle d'accès garantissent que les sujets ne peuvent observer ou modifier des objets que s'ils ont un niveau de sécurité approprié, assurant ainsi l'intégrité et la confidentialité des données dans les systèmes informatiques.

Le modèle de Biba 1977

Les politiques obligatoires sont des règles de sécurité développées pour maintenir l'intégrité des données dans les systèmes informatiques. Elles appliquent à l'intégrité un modèle similaire à celui de Bell-LaPadula, qui est utilisé pour la confidentialité.

Selon ces politiques, chaque sujet se voit attribuer un niveau d'intégrité (is(s)), correspondant à la confiance accordée à ce sujet, et chaque objet possède un niveau d'intégrité (io(o)), correspondant au niveau d'intégrité du sujet qui l'a créé. Les objectifs de sécurité de cette politique visent à :

Interdire toute propagation d'information d'un objet situé à un certain niveau d'intégrité vers un objet situé à un niveau d'intégrité supérieur. Cela garantit que les données ne peuvent pas être modifiées ou altérées par des sujets de confiance inférieure, préservant ainsi leur intégrité.

Interdire à tout sujet situé à un certain niveau d'intégrité de modifier un objet possédant un niveau d'intégrité supérieur. Cela garantit que les sujets de confiance inférieure ne peuvent pas altérer les données créées par des sujets de confiance supérieure, préservant ainsi leur intégrité et évitant toute altération ou corruption non autorisée.

Le modèle de Biba 1977 (2)

Le schéma d'autorisation en matière de sécurité découle de la recherche de certaines propriétés, en tenant compte des labels d'intégrité et du regroupement des opérations en trois classes : observation, modification et invocation.

Un sujet ne peut modifier un objet que si le label d'intégrité du sujet domine le label d'intégrité de l'objet :

$$(si, oj, modifier) \Rightarrow io(oj) \leq is(si).$$

Cette règle garantit que seuls les sujets dont le niveau d'intégrité est au moins aussi élevé que celui de l'objet peuvent modifier cet objet, préservant ainsi l'intégrité des données.

Un sujet ne peut observer un objet que si le label d'intégrité de l'objet domine le label d'intégrité du sujet :

$$(si, oj, observer) \Rightarrow is(si) \leq io(oj).$$

Cette règle assure que seuls les sujets dont le niveau d'intégrité est inférieur ou égal à celui de l'objet peuvent observer cet objet, évitant ainsi toute divulgation non autorisée d'informations sensibles.

Un sujet ne peut invoquer un sujet j que si le label d'intégrité de si domine le label d'intégrité de sj :

$$(si, sj, invoquer) \Rightarrow is(sj) \leq is(si).$$

Cette règle garantit que seuls les sujets dont le niveau d'intégrité est au moins aussi élevé que celui du sujet invoqué peuvent effectuer cette action, préservant ainsi la sécurité du système en limitant les accès aux sujets de confiance suffisante.

Le modèle Clark et Wilson

Dans l'environnement commercial, la prévention contre les modifications non autorisées de l'information, notamment contre les fraudes et les erreurs, est d'une importance capitale. Même si un utilisateur est autorisé à manipuler certaines données, cela ne doit pas entraîner de perte ou de falsification des actifs, tels que les capitaux et les informations.

La politique de Clarke-Wilson repose sur deux principes bien connus :

Transactions bien formées : Ce principe stipule que les utilisateurs n'ont pas le droit de manipuler les données de manière arbitraire, mais seulement à travers des procédures de transformation spécifiques qui préservent l'intégrité des données. Les actions des utilisateurs doivent suivre des protocoles prédéfinis et autorisés pour garantir que les données restent exactes et fiables.

Séparation des pouvoirs : Aussi connue sous le nom de "separation of duty", ce mécanisme de contrôle des fraudes et des erreurs est basé sur la répartition des opérations entre plusieurs parties et l'attribution de droits différents mais complémentaires à différentes catégories de personnes. En séparant les responsabilités et les autorisations entre plusieurs individus ou entités, ce principe réduit les risques de fraudes et d'erreurs en obligeant plusieurs personnes à s'impliquer dans un processus de validation ou de transaction.

Le modèle Clark et Wilson (2)

Dans ce modèle, plusieurs concepts clés doivent être définis pour assurer la sécurité des données :

1.Les données contraintes (CDI) : Ces données représentent les éléments de données dont nous voulons préserver l'intégrité. Elles sont soumises à des contraintes et à des mécanismes de protection pour garantir qu'elles ne sont pas altérées de manière non autorisée.

2.Les données non contraintes (UDI) : Ces données représentent les éléments dont l'intégrité n'est pas garantie et qui peuvent être manipulés de manière arbitraire. Elles ne sont pas soumises aux mêmes contraintes de protection que les CDI.

3.Procédures pour la vérification de l'intégrité (IVP) : Ces procédures sont utilisées pour confirmer que tous les CDI dans les systèmes sont conformes aux spécifications d'intégrité au moment où les IVP sont exécutées. Elles permettent de s'assurer que les données contraintes restent intactes et non altérées.

4.Procédures de transformation (TP) : Correspondant au concept de transactions bien formées, ces procédures visent à changer les CDI d'un état valide à un autre état valide. Elles permettent des modifications contrôlées et autorisées des données contraintes tout en préservant leur intégrité et leur fiabilité.

Le modèle Clark et Wilson (3)

Dans le modèle de sécurité de Clarke-Wilson, les règles appliquées dans leur politique de sécurité sont définies de la manière suivante :

L'ensemble des utilisateurs, noté $U=\{u_1, u_2, \dots\}$.

L'ensemble des données contraintes, $CDI=\{CDI_1, CDI_2, \dots\}$.

L'ensemble des données non contraintes, $UDI=\{UDI_1, UDI_2, \dots\}$.

L'ensemble des opérations de vérification de l'intégrité des données, $IVP=\{IVP_1, IVP_2, \dots\}$. Ainsi, les CDI sont certifiées par des IVP.

L'ensemble des opérations de transformation des données (Transformation Procedures), noté $TP=\{TP_1, TP_2, \dots\}$. Les CDI ne peuvent être manipulées que par des TP.

Une relation R_c liant chaque opération de transformation TP_i à un sous-ensemble c de CDI ; R_c précise les données que l'opération peut manipuler.

Une relation R_u liant chaque utilisateur u et chaque opération de transformation TP_i à un sous-ensemble de CDI ; R_u précise les données contraintes qu'un utilisateur est autorisé à manipuler (par le biais de TP_i).

Le modèle Clark et Wilson (4)

Les règles obligatoires du modèle de Clarke-Wilson imposent plusieurs contraintes :

Les CDI doivent être dans un état valide, vérifié par les IVP. Cela garantit que les données contraintes sont toujours fiables et conformes aux spécifications d'intégrité.

Toutes les opérations de TP doivent être certifiées. Si une donnée contrainte est valide avant l'exécution d'une opération TP_i , elle doit rester valide après l'exécution de TP_i . De plus, les opérations de TP ne peuvent être effectuées que sur des données contraintes spécifiées par R_c .

Chaque utilisateur n'effectue des opérations de transformations sur des données contraintes que si celles-ci lui sont associées par R_u . Cela garantit que chaque utilisateur ne peut manipuler que les données auxquelles il est autorisé.

Le modèle Clark et Wilson (5)

La relation Ru doit être certifiée afin de refléter les besoins de la séparation des pouvoirs. Cela assure que les droits et les autorisations des utilisateurs sont correctement définis et respectés.

Le système doit authentifier l'identité de chaque utilisateur souhaitant exécuter une opération de TP. Cela garantit que seuls les utilisateurs autorisés peuvent effectuer des opérations sur les données.

Avant d'être exécutées, les opérations de TP doivent enregistrer leurs identifiants dans les journaux d'audit, qui sont considérés comme des données contraintes. Cela permet de garder une trace des actions effectuées sur les données et de garantir la traçabilité des opérations.

Chaque opération TP qui accepte une donnée UDI en entrée doit garantir que, quelle que soit la valeur de UDI, elle n'effectue que des transformations conduisant à une donnée contrainte valide (CDIk) ou rejette UDI si elle ne peut pas être transformée de manière valide. Cela assure que seules les données contraintes sont manipulées de manière sécurisée dans le système.

Le modèle Muraille de chine

Le modèle de Brewer et Nash, également connu sous le nom de modèle de la muraille de Chine, est un modèle de contrôle d'accès qui fournit un contrôle d'accès à l'information pouvant évoluer dynamiquement.

Ce modèle a été développé pour réduire les conflits d'intérêts dans des organisations commerciales et assurer la propriété de la confidentialité. Un conflit d'intérêts (COI) survient lorsqu'un sujet accède aux informations confidentielles de deux organisations en compétition, permettant ainsi un flux d'information d'une organisation vers une autre.

Les concepts clés de ce modèle incluent :

- **Objet** : qui est un élément appartenant à une organisation.
- **Organisation** : qui dispose de plusieurs objets utilisés dans le cadre de ses services.
- **Classe de conflits d'intérêts** : qui regroupe des organisations qui sont en concurrence les unes avec les autres.

Le modèle Muraille de chine (2)

Dans ce modèle, le système classe l'information en trois niveaux hiérarchiques :

1.Au niveau le plus bas, on trouve les données simples, telles que les fichiers, qui sont les objets du système.

2.Au niveau intermédiaire, les objets sont regroupés selon la compagnie à laquelle ils appartiennent, formant des ensembles de données (CD).

3.Au niveau le plus élevé, les compagnies sont regroupées en fonction de leur compétitivité ou de leur absence de compétitivité. Cela forme les classes de conflit d'intérêt (COD) mentionnées précédemment.

Les auteurs introduisent une matrice N qui contient l'historique des accès précédemment effectués. Il s'agit de placer les sujets en lignes et les objets en colonnes, où chaque case contient un booléen indiquant si un accès a déjà eu lieu pour le couple (sujet, objet) correspondant. Ainsi, deux règles régissent les accès entre sujets et objets :

•**Simple property** : L'accès en lecture à un objet est autorisé si :

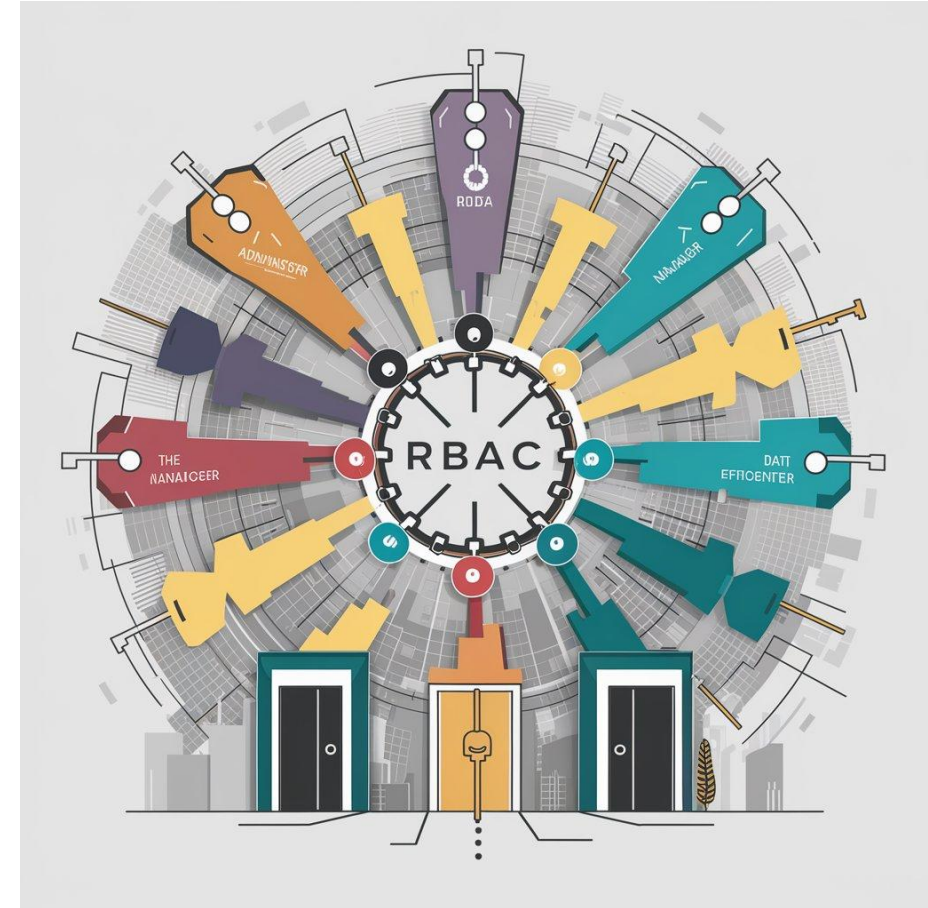
- Il appartient à la même compagnie qu'un objet déjà accédé.
- Ou s'il appartient à une autre classe de conflit d'intérêt que les objets déjà en accès.

•**Étoile (*)-property** : L'accès en écriture à un objet est autorisé si :

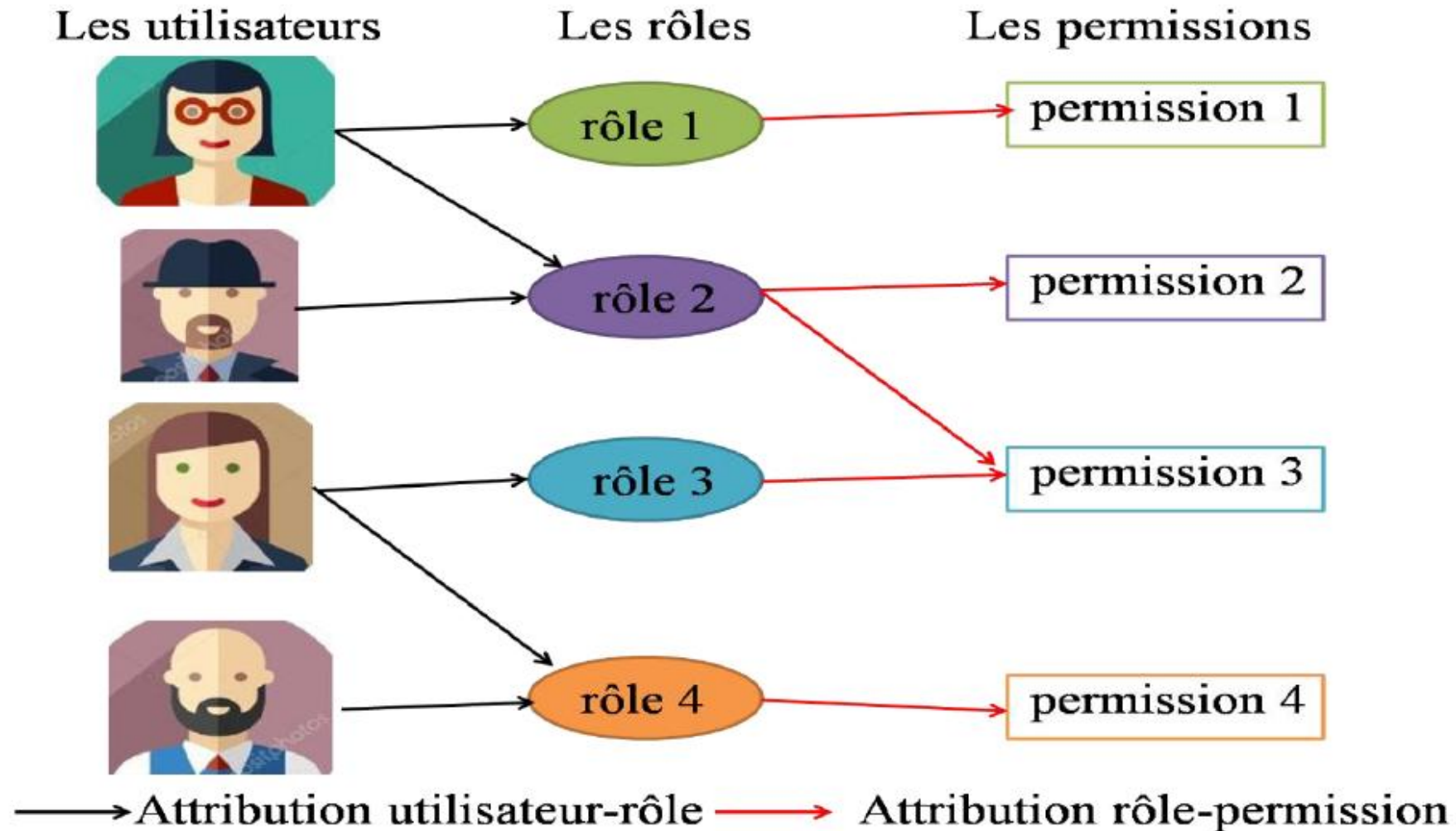
- L'accès est autorisé par la règle de sécurité simple.
- Et si le sujet ne peut pas lire aucun objet d'une autre compagnie contenant des informations non stérilisées.

Le modèle de contrôle d'accès basé sur le rôle (RBAC)

- Il a été présenté comme une alternative au DAC et au MAC. Le RBAC (pour Contrôle d'Accès Basé sur les Rôles) [Ferraiolo&Kuhn,1992],[Sandhu et al.,1996]) est un modèle de contrôle d'accès qui se concentre sur les rôles.
- Chaque rôle représente de manière abstraite une fonction identifiée dans l'organisation, telle que chef de service, ingénieur d'étude, etc.
- À chaque rôle sont associées des permissions, qui sont représentées par des droits d'accès.
- Une permission est un ensemble de droits correspondant aux tâches pouvant être effectuées par un rôle.



Le modèle de contrôle d'accès basé sur le rôle (RBAC) (2)



Le modèle de contrôle d'accès basé sur le rôle (RBAC) (3)

Par exemple: soit l'utilisateur U1 qui possède le rôle « directeur d'hôpital », les permissions qui sont attribuées à ce rôle sont : (gérer, soigner, acheter). De plus, gérer l'accès à un système utilisant ce modèle nécessite le respect de certaines politiques de contrôles d'accès qui sont faites à base de rôle, par exemple : Pour qu'un utilisateur puisse accéder aux informations médicales d'un patient de l'hôpital, il doit avoir le rôle « médecin ». De cette manière l'utilisation du rôle offre une indépendance logique dans la spécification des autorisations d'utilisateur.

Le modèle de contrôle d'accès basé sur le rôle (RBAC) (4)

En outre, cette indépendance simplifie la gestion de la politique de sécurité car lorsqu'un nouvel utilisateur rejoint l'hôpital, il suffit de lui attribuer un rôle approprié sans modifier les permissions elles-mêmes. Par exemple, si un nouvel employé est recruté comme médecin, on lui assigne simplement le rôle « médecin » pour qu'il hérite automatiquement des permissions associées, comme « soigner » ou « consulter les dossiers médicaux ». Ce mécanisme rend le système plus flexible, évolutif et cohérent, en séparant clairement les responsabilités des utilisateurs des droits d'accès qu'ils exercent sur les ressources du système.

Le modèle de contrôle d'accès basé sur le rôle (RBAC) (5)

- En 1992, Ferraiolo et Kuhn ont développé le premier modèle RBAC [FK92].
- En 1995, ils l'ont bien raffiné en définissant tous les termes du modèle RBAC et ont validé son utilisation et sa faisabilité [FCK95].
- En 1996, Sandhu et al. [SCFY96] proposent la famille la plus connue des modèles RBAC.
- En 2001, NIST a proposé un modèle standard pour RBAC.
- En 2004, ANSI (The American National Standard Institute) a proposé le standard ANSI RBAC3 en se basant sur [SFK00] et [FSG+01]. Ce standard définit un modèle RBAC de référence et fournit les caractéristiques que tout système RBAC doit avoir.

Le modèle de contrôle d'accès basé sur le rôle (RBAC) (6)

Ce modèle repose sur plusieurs concepts clés qui définissent les interactions entre les utilisateurs, les rôles, les objets, les opérations et les permissions. Comprendre ces termes est essentiel pour appréhender le fonctionnement et la mise en œuvre efficace du RBAC dans divers environnements informatiques.

Utilisateur: Dans le contexte du système, un utilisateur est défini comme une personne utilisant les fonctionnalités offertes.

Rôle: Le rôle se réfère à la position ou à la responsabilité qu'un individu peut occuper ou assumer au sein de l'organisation.

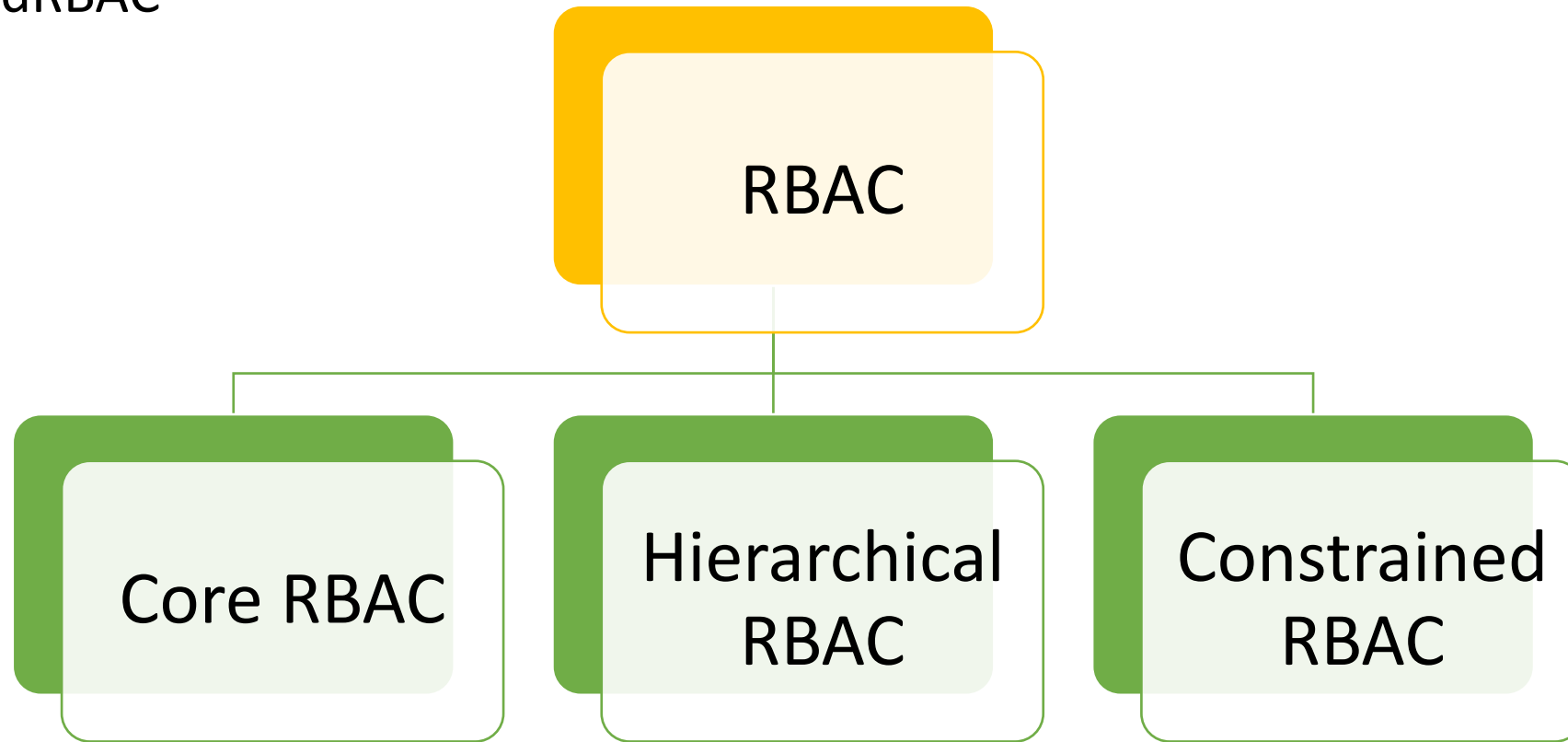
Objet: Tout élément représentant une ressource du système, tel qu'un fichier, une imprimante ou une base de données, est désigné comme un objet.

Opération: Une opération consiste en une action exécutable par le système en réponse à une demande de l'utilisateur.

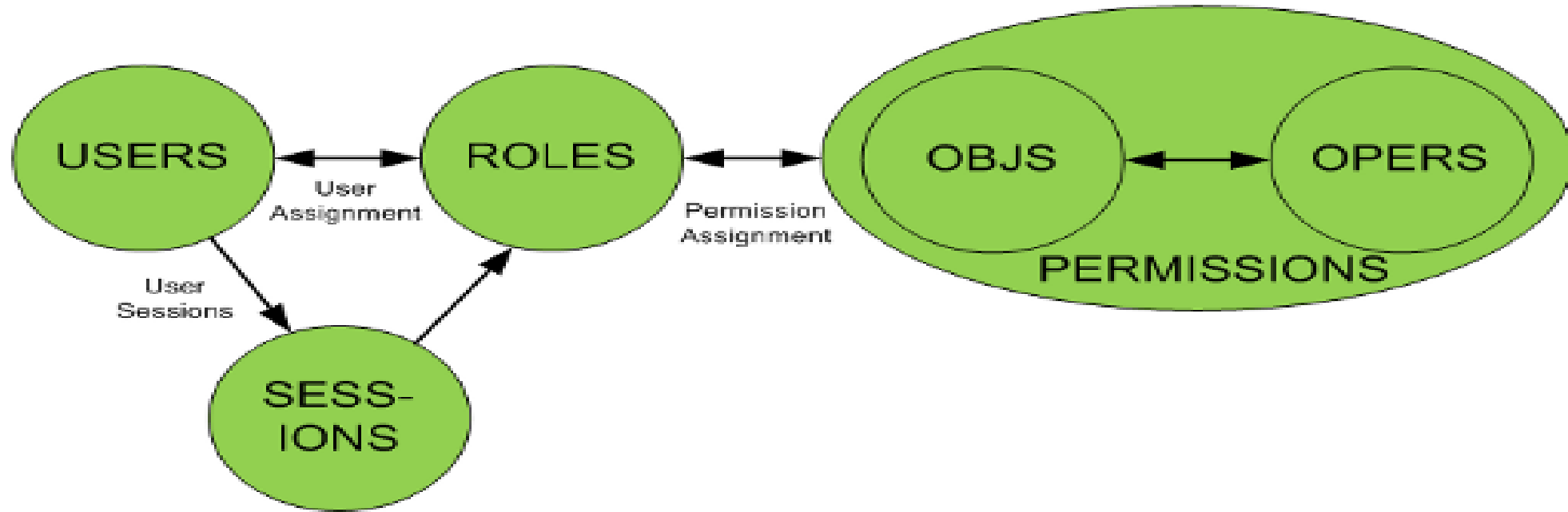
Permission: Une permission correspond à l'autorisation accordée pour effectuer une ou plusieurs opérations sur un objet donné.

Le modèle de contrôle d'accès basé sur le rôle (RBAC) (7)

Trois modèles de référence ont été définis dans le standard : CoreRBAC, HierarchicalRBAC et le ConstrainedRBAC



Core RBAC



Dans ce modèle de référence, on attribue des **rôles** aux **utilisateurs**, et ces rôles déterminent les autorisations qu'ils ont concernant divers **objets** et **opérations** au sein d'un système. Les **sessions** sont associées aux utilisateurs, indiquant des instances individuelles d'interaction de l'utilisateur avec le système.

Le modèle RBAC est une approche courante pour gérer les autorisations des utilisateurs au sein des systèmes logiciels, offrant un moyen de restreindre l'accès au système aux utilisateurs autorisés.

HierarchicalRBAC

Le **HierarchicalRBAC** introduit des hiérarchies de rôles permettant de structurer les rôles afin de refléter les niveaux d'hiérarchie dans une organisation.

- Une hiérarchie de rôles définit des relations d'héritage entre les rôles.
- Un rôle $r1$ hérite d'un rôle $r2$ si tous les privilèges de $r2$ sont aussi des privilèges de $r1$.
- Le standard définit deux types de hiérarchies de rôles : des hiérarchies de rôles générales et des hiérarchies de rôles limitées.
- Une hiérarchie de rôles générale est un ordre partiel arbitraire représentant l'ensemble des rôles R . Cette hiérarchie supporte le concept d'héritage multiple qui offre la possibilité d'hériter des permissions de deux ou plusieurs rôles dans le système. De manière formelle, nous définissons un ordre partiel noté $\geq$ sur l'ensemble des rôles tel que :
 - étant donné deux rôles $r1$ et $r2$, $r1 \geq r2$ implique $\text{authorized-permissions}(r2) \subseteq \text{authorized-permissions}(r1)$ et $\text{authorized-users}(r1) \subseteq \text{authorized-users}(r2)$.
- La hiérarchie des rôles limitée impose la définition des rôles sous forme d'un arbre plus simple où un rôle ne peut avoir qu'un seul descendant.

HierarchicalRBAC (2)

Prenons un exemple pour illustrer cela. Supposons que nous ayons deux rôles dans un système de gestion de contenu : "Éditeur" et "Administrateur".

Le rôle "Éditeur" a des permissions pour créer, modifier et supprimer des articles sur le site web.

Le rôle **Administrateur** a des permissions pour effectuer toutes les actions des éditeurs ainsi que des permissions supplémentaires pour gérer les utilisateurs et les paramètres du site.

Dans cet exemple, nous avons un ordre partiel où le rôle "Administrateur" est supérieur au rôle **Éditeur**. Cela signifie que toutes les permissions autorisées pour un éditeur sont également autorisées pour un administrateur. De plus, tous les utilisateurs autorisés en tant qu'éditeurs seront également autorisés en tant qu'administrateurs.

En d'autres termes, si $r1$ représente le rôle "Administrateur" et $r2$ représente le rôle "Éditeur", alors $r1 \geq r2$ implique que les permissions autorisées pour "Éditeur" ($\text{authorized-permissions}(r2)$) doivent être un sous-ensemble des permissions autorisées pour "Administrateur" ($\text{authorized-permissions}(r1)$). De même, les utilisateurs autorisés en tant qu'éditeurs ($\text{authorized-users}(r2)$) doivent être un sous-ensemble des utilisateurs autorisés en tant qu'administrateurs ($\text{authorized-users}(r1)$).

ConstrainedRBAC

Le ConstrainedRBAC (Contrôle d'Accès Basé sur les Rôles avec Contraintes) introduit des règles spécifiques lors de l'attribution des rôles aux utilisateurs et lors de l'activation de ces rôles. Les contraintes de séparation de droit, également connues sous le nom de Separation of Duty Constraints (SoD), sont l'un des aspects clés de ce modèle.

Ces contraintes sont mises en place dans le but d'empêcher les utilisateurs d'acquérir un niveau excessif d'autorité ou de privilèges, et également de limiter les conflits potentiels qui pourraient survenir entre les politiques de sécurité du système.

ConstrainedRBAC (2)

Par exemple, considérons un système dans lequel un utilisateur est autorisé à approuver les transactions financières et également à valider les demandes d'accès aux données sensibles. Ces deux responsabilités, bien que distinctes, peuvent entraîner des conflits d'intérêts ou des risques de fraude si elles sont détenues par la même personne.

Les contraintes de séparation de droit dans le cadre du ConstrainedRBAC pourraient alors être utilisées pour spécifier qu'un utilisateur ne peut pas avoir à la fois le pouvoir d'approuver les transactions financières et celui de valider les demandes d'accès aux données sensibles. En imposant cette contrainte, le système garantit que les responsabilités sont réparties de manière appropriée et que les risques de conflits d'intérêts ou de fraudes sont réduits.

ConstrainedRBAC (3)

Pour formaliser les contraintes de sécurité, deux types de relations ont été établis : la séparation de droits statique (Static Separation of Duty - SSD) et la séparation de droits dynamique (Dynamic Separation of Duty - DSD).

Les contraintes statiques (SSD) servent à limiter l'attribution des rôles aux utilisateurs. Une fois qu'un utilisateur se voit attribuer un rôle R1, il lui est interdit de recevoir d'autres rôles susceptibles de créer des conflits avec le rôle R1. Ces contraintes sont largement employées dans divers contextes pour garantir une distribution équilibrée des responsabilités et prévenir les conflits d'intérêts ou de privilèges.

Les contraintes dynamiques (DSD) visent à éviter les conflits lors de l'activation des rôles. Elles autorisent un utilisateur à activer simultanément un ou plusieurs rôles, à condition que cette activation ne génère pas de conflit. Par exemple, un utilisateur pourrait se voir accorder les rôles d'éditeur et d'administrateur en même temps, pour autant que ces deux rôles ne s'opposent pas dans le système. Les contraintes DSD sont particulièrement utiles pour permettre une gestion flexible des autorisations tout en préservant la sécurité du système.

Exemples SSD et DSD

Supposons que nous ayons un système de gestion de contenu en ligne avec deux types d'utilisateurs : les éditeurs et les administrateurs.

❑ **Contraintes de séparation des droits statique (SSD) :**

- Un éditeur peut modifier et publier du contenu sur le site.
- Un administrateur peut non seulement modifier et publier du contenu, mais aussi gérer les utilisateurs et les autorisations.
- Selon la contrainte SSD, un utilisateur qui est désigné comme éditeur ne peut pas se voir attribuer le rôle d'administrateur simultanément. Cela garantit une séparation claire des responsabilités et évite les conflits potentiels entre les tâches éditoriales et administratives.

❑ **Contraintes de séparation des droits dynamique (DSD) :**

- Un utilisateur peut être à la fois modérateur et administrateur.
- Un modérateur a le pouvoir de gérer les commentaires et le contenu utilisateur, tandis qu'un administrateur a des privilèges plus étendus, y compris la gestion des utilisateurs et des paramètres du site.
- La contrainte DSD permet à un utilisateur d'activer simultanément les rôles de modérateur et d'administrateur, tant que ces rôles ne présentent pas de conflits fonctionnels. Par exemple, un utilisateur pourrait être autorisé à modérer les commentaires tout en ayant des droits d'administration limités, mais il ne pourrait pas être à la fois modérateur et éditeur, car ces deux rôles pourraient entrer en conflit dans la gestion du contenu.

Le modèle de contrôle d'accès basé sur l'organisation (Or-BAC)

OrBAC, qui a été introduit pour la première fois en 2003, s'inspire des principes des modèles de contrôle d'accès basés sur les rôles (RBAC), mais va plus loin en offrant la possibilité de modifier dynamiquement la politique de sécurité en fonction du contexte. Il est centré sur le concept d'organisation.

OrBAC permet l'expression de permissions, d'obligations et d'interdictions qui dépendent du contexte. Il établit deux niveaux complets d'abstraction :

- ❖ **Niveau abstrait:** Ce niveau permet de décrire la politique de sécurité à l'aide d'entités abstraites, sans se soucier de la façon dont chaque organisation les met en œuvre.
- ❖ **Niveau concret:** À ce niveau, les requêtes d'accès des sujets sont traitées en leur accordant les privilèges en fonction des règles qui définissent l'organisation, le rôle du sujet, l'action requise (c'est-à-dire l'activité), l'objet cible (c'est-à-dire la vue) et le contexte.

Le modèle de contrôle d'accès basé sur l'organisation (Or-BAC) (3)

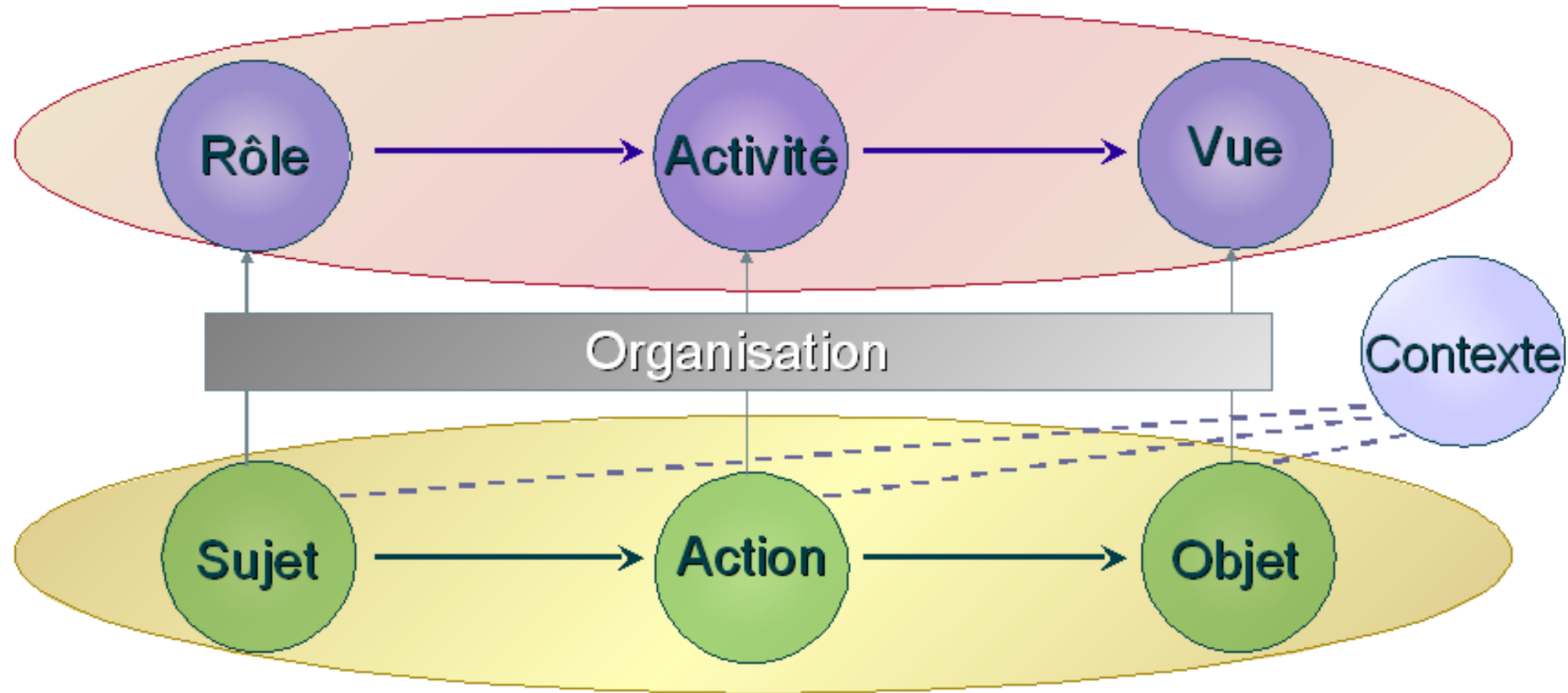
Un **rôle** représente de manière abstraite un groupe d'utilisateurs exerçant une fonction dans une organisation.

Une **activité** représente de manière abstraite une ou plusieurs actions pouvant être effectuées.

Une **vue** correspond à un ensemble d'objets, et la relation Utilise() spécifie quelles activités sont autorisées à accéder à ces vues.

Les relations **Habilite()**, **Utilise()** et **Considere()** sont utilisées pour associer les entités abstraites aux entités concrètes dans le contexte de la politique de sécurité d'une organisation.

Permission d'un rôle de réaliser une activité sur une vue



https://fr.wikipedia.org/wiki/Contr%C3%B4le_d%27acc%C3%A8s_bas%C3%A9_sur_l%27organisation#/media/Fichier:OrBACschemaFR.png

Concept de rôle et relation Habilité ()

Le concept de rôle et la relation Habilité() sont des éléments essentiels dans la politique de sécurité des organisations. Voici une explication détaillée de ces concepts :

➤ Concept de rôle

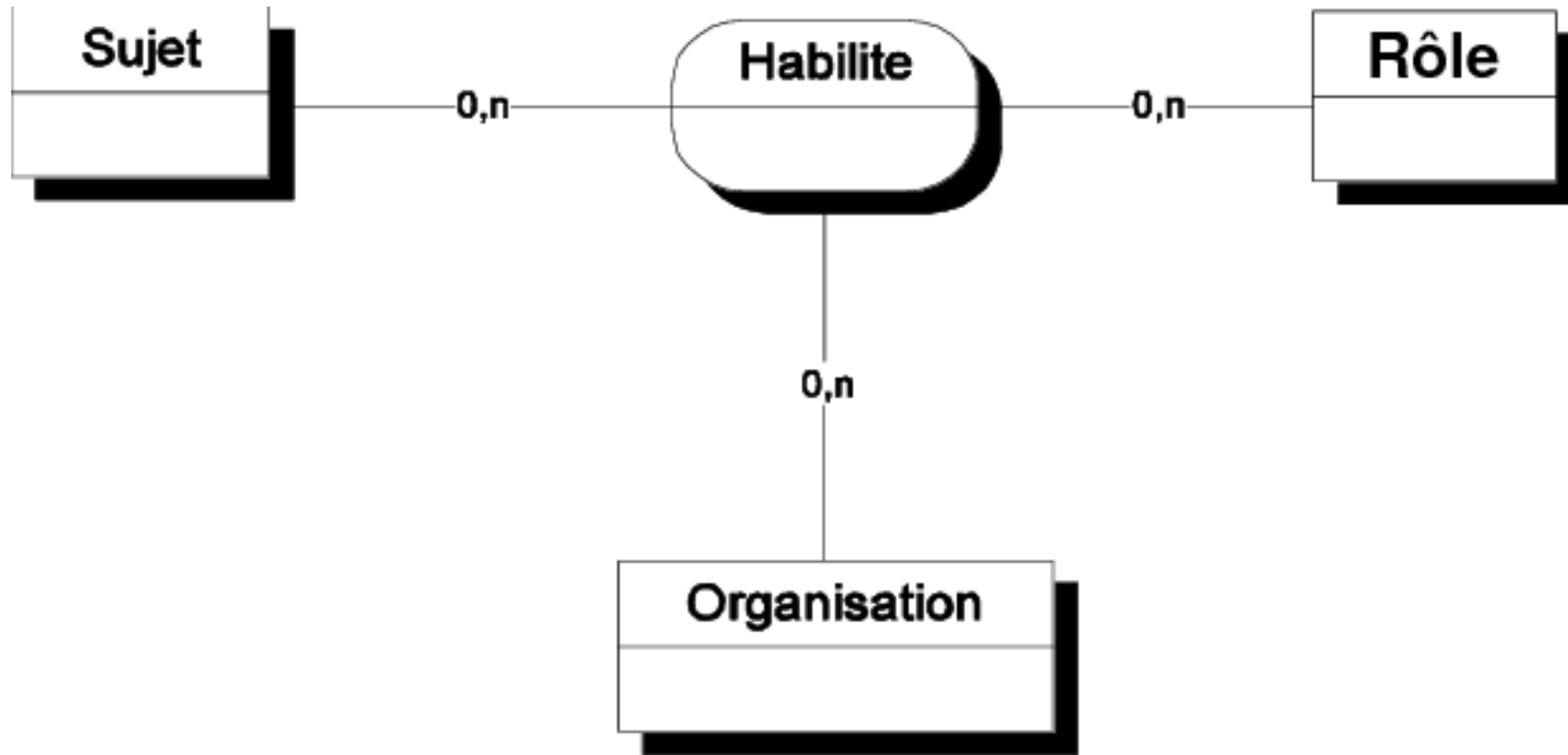
- Un rôle représente une fonction, une position ou un ensemble de privilèges au sein d'une organisation.
- Les rôles définissent les responsabilités et les autorisations des utilisateurs ou des entités organisationnelles.
- Ils permettent de catégoriser les utilisateurs en fonction de leurs fonctions ou de leurs responsabilités.

Par exemple: Dans un département informatique, les rôles peuvent inclure "administrateur système", "développeur", "analyste de données", "gestionnaire de réseau", "testeur de logiciel", etc.

➤ Relation Habilité()

- La relation Habilité(Org, s, r) est un prédicat qui décrit l'attribution d'un rôle à un sujet (utilisateur ou organisation) au sein d'une organisation spécifique (Org).
- Cette relation indique que le sujet s est habilité au rôle r dans l'organisation Org.
- Elle est utilisée pour définir les autorisations et les responsabilités des utilisateurs ou des entités organisationnelles dans le contexte d'une organisation donnée.

Concept de rôle et relation Habilité () (2)



https://www.researchgate.net/figure/La-relation-Habilite-Contrairement-aux-modeles-TMAC-et-RBAC-qui-ne-considerent-que-des_fig3_271636704

Concept de vue et relation utilise()

La relation **Utilise()** est une relation qui associe un **objet** à une **vue** au sein d'une organisation. Elle est définie par le prédicat $Utilise(Org, O, V)$, où Org représente l'organisation, O représente l'objet et V représente la vue. Cette relation indique que, au **sein de l'organisation donnée, l'objet est utilisé dans le contexte de la vue spécifiée**.

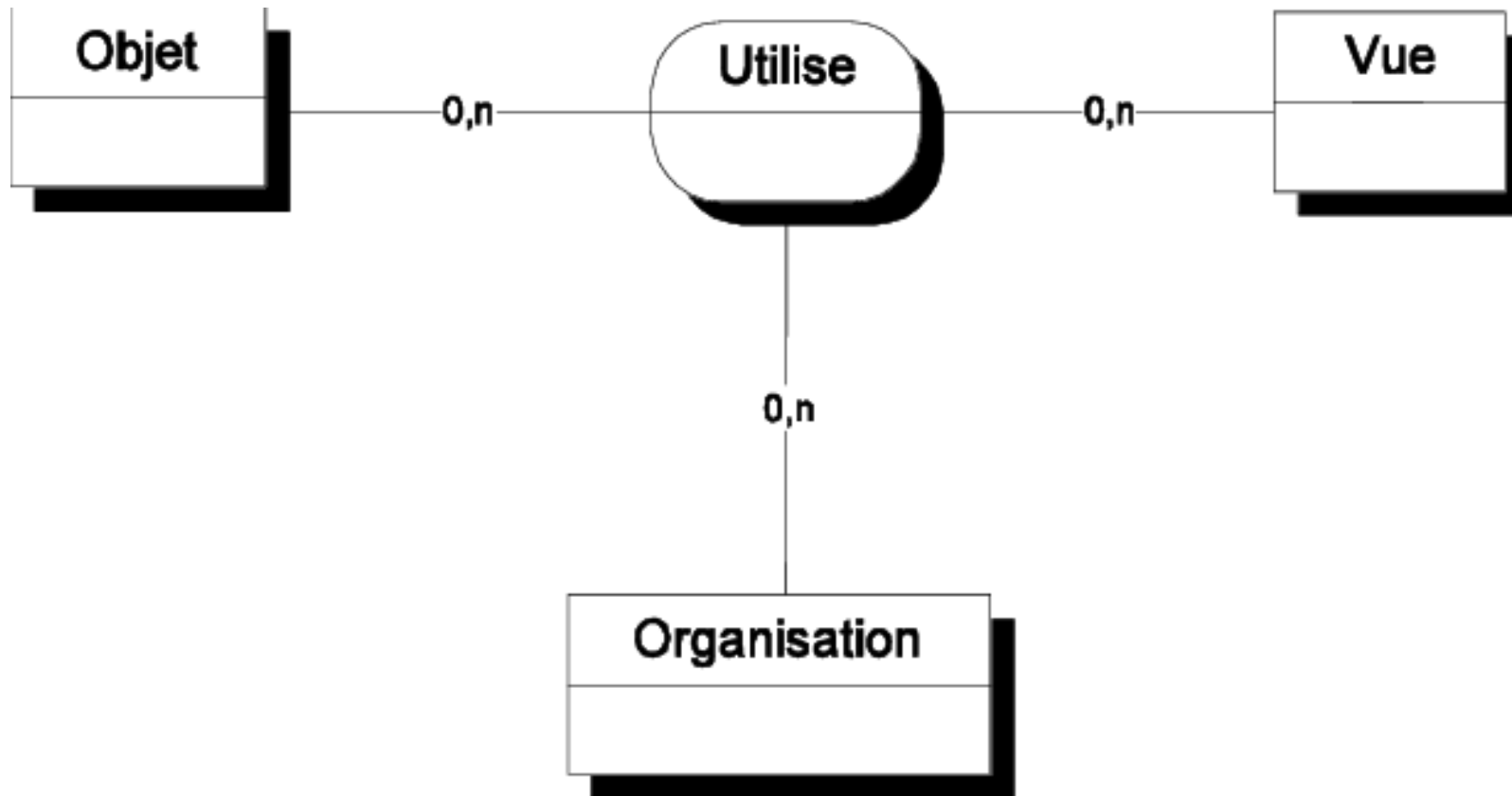
Exemple:

Dans un contexte de gestion immobilière, la relation $Utilise()$ peut être appliquée pour spécifier l'utilisation de fichiers et de données associées à des propriétés spécifiques. Par exemple, considérons une agence immobilière nommée "ImmoPlus" qui utilise un fichier Excel nommé "Liste des propriétés disponibles.xlsx" pour répertorier toutes les propriétés disponibles sur le marché.

La relation $Utilise()$ peut être utilisée pour déclarer que ce fichier Excel est utilisé comme une vue nommée "Vue des propriétés disponibles" au sein de l'agence ImmoPlus. Ainsi, nous pouvons exprimer cette relation comme suit : $Utilise(ImmoPlus, Liste\ des\ propriétés\ disponibles.xlsx, Vue\ des\ propriétés\ disponibles)$.

De manière similaire, cette relation peut également être utilisée pour spécifier l'utilisation d'une base de données relationnelle contenant des informations sur les propriétés. Par exemple, $Utilise(ImmoPlus, Base\ de\ données\ propriétés, Vue\ des\ propriétés\ disponibles)$ signifierait que l'agence immobilière ImmoPlus **utilise une base de données relationnelle nommée "Base de données propriétés" comme source d'informations** pour la vue des propriétés disponibles.

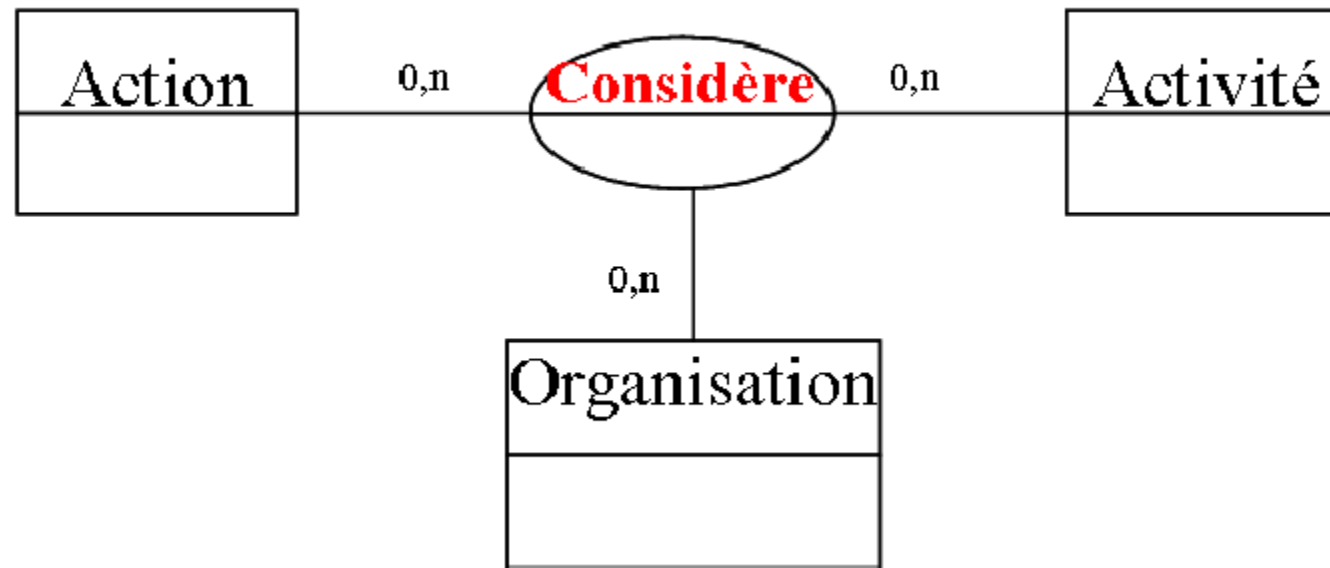
Concept de vue et relation utilise() (2)



https://www.researchgate.net/figure/La-relation-Utilise_fig4_271636704

Concept d'activité et relation Considère ()

La relation **Considère()** est une relation qui associe une **action** à une **activité** au sein d'une organisation. Elle est définie par le prédicat $\text{Considère}(\text{Org}, \text{ac}, \text{ay})$, où Org représente l'organisation, ac représente l'action et ay représente l'activité. Cette relation indique que, **au sein de l'organisation donnée, l'action ac est considérée comme faisant partie de l'activité ay.**



Exemple sur le concept d'activité et relation Considère ()

Supposons une entreprise de développement de logiciels nommée "TechSoft". Dans cette entreprise, la relation Considère() peut être utilisée pour attribuer des actions à des activités spécifiques.

Par exemple, considérons une action nommée "codage" qui fait partie de l'activité "développement". La relation Considère() serait exprimée comme suit : Considère(TechSoft, codage, développement).

De manière similaire, l'action "test" pourrait être considérée comme faisant partie de l'activité "assurance qualité" dans cette entreprise. Ainsi, nous pourrions avoir : Considère(TechSoft, test, assurance qualité).

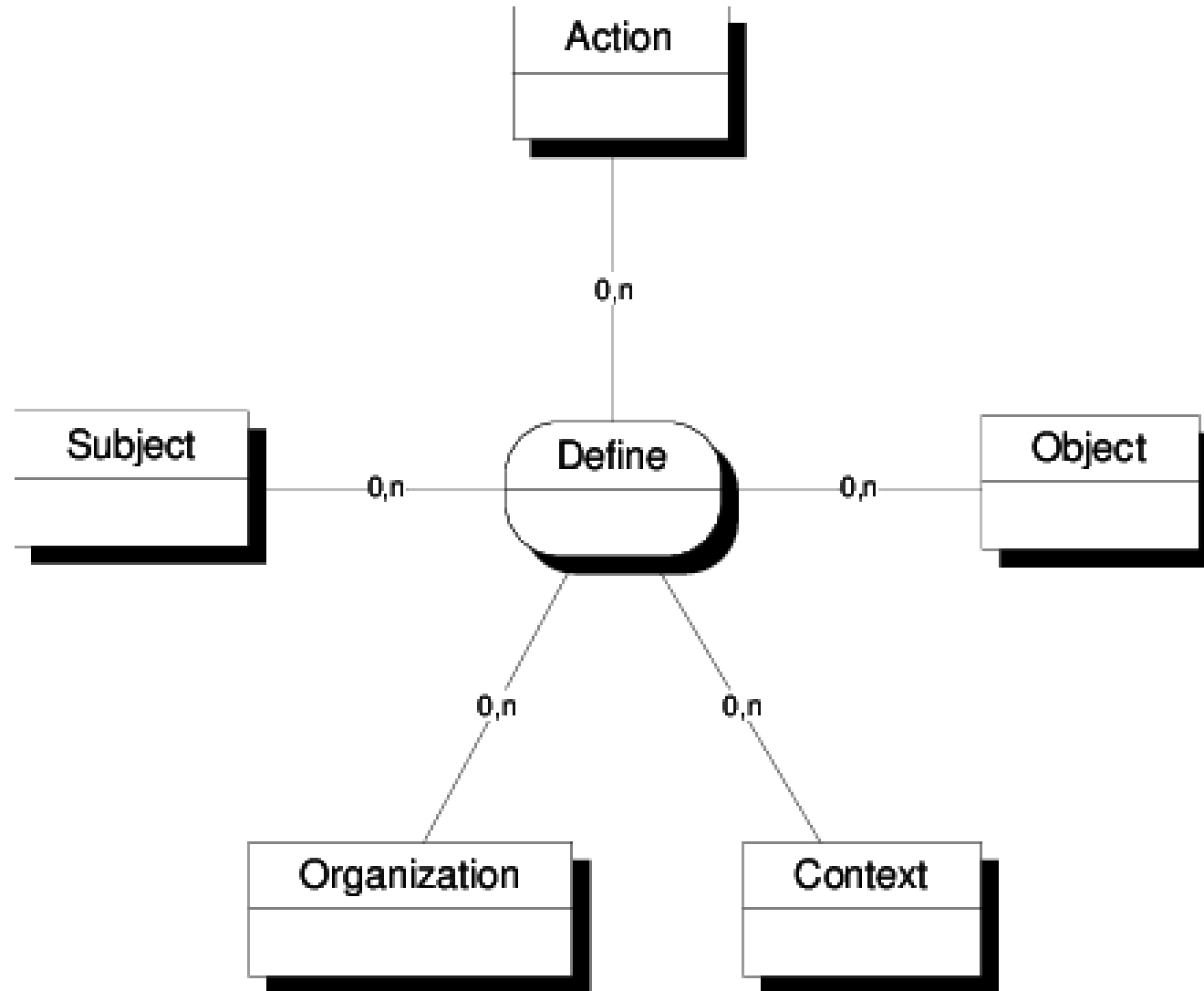
Cette relation entre les actions et les activités permet à l'entreprise de mieux organiser ses processus de développement de logiciels et de gérer efficacement les tâches associées à chaque activité.

Concept de contexte et relation Définit()

Le **contexte** dans le domaine de la sécurité peut être défini comme toute **information qui caractérise la situation d'une entité** ou qui spécifie les circonstances concrètes dans lesquelles les organisations accordent des permissions à des rôles pour réaliser des activités sur des vues. Par exemple, le contexte peut inclure des informations telles que quand l'utilisateur a-t-il le droit d'accéder à une information et d'où l'accès est-il possible.

La relation Définit(Org, s, o, ac, c) est un prédicat qui rassemble les éléments concrets de la tâche ciblée par la requête d'accès. Au sein de l'organisation donnée (Org), l'accès est défini par les sujets (s) réalisant l'action (ac) sur l'objet (o) en présence du contexte (c).

Concept de contexte et relation Définir() (2)



<https://www.researchgate.net/profile/Yves-Deswarte/publication/4020429/figure/fig4/AS:340377363075089@1458163695666/The-Def-ine-relationship.png>

Exemple sur le concept de contexte et relation Définir()

Dans un contexte universitaire, la relation Définir(Org, s, o, ac, c) pourrait être utilisée pour définir les autorisations d'accès des étudiants à des documents académiques en ligne. Par exemple :

Supposons que l'organisation soit une université nommée "Université Badji Mokhtar". Une étudiante nommée « Nadine » (sujet) souhaite accéder (action) à un document de recherche (objet) intitulé "Projet de recherche sur l'intelligence artificielle" (o) pour effectuer une analyse (contexte). Dans ce contexte, le prédicat Définir(Université Badji Mokhtar, Nadine, accéder, Projet IA, analyse) pourrait être utilisé pour spécifier que l'étudiante Nadine est autorisée à accéder au document pour effectuer une analyse, dans le cadre de ses études en informatique.

De manière similaire, un autre étudiant nommé « Rassim » pourrait avoir des autorisations différentes en fonction de son contexte. Par exemple, Définir(Université Badji Mokhtar, Rassim, accéder, Projet IA, rédaction de rapport) pourrait signifier que Rassim est autorisé à accéder au même document pour rédiger un rapport, dans le cadre d'un cours de science des données.

Règles de sécurité

$\text{org} \in \text{Org}, s \in S, \alpha \in \text{Actions}, o \in O, r \in R, a \in \text{Activités},$
 $v \in V, c \in C$

$\text{Permission}(\text{org}, r, v, a, c) \wedge$

$\text{Habilité}(\text{org}, s, r) \wedge$

$\text{Considéré}(\text{org}, \alpha, a) \wedge$

$\text{Utilise}(\text{org}, o, v) \wedge$

$\text{Définit}(\text{org}, s, \alpha, o, c)$

$\rightarrow \text{Est_permis}(s, \alpha, o)$

Règles de sécurité (2)

Le prédicat `Est_Permis(s, o, ac)` (et ses variations `Est_Interdit()`, `Est_Recommandé()`, `Est_Obligé()`) est un prédicat utilisé pour définir les règles d'accès à un niveau concret. Il spécifie si un sujet est autorisé à réaliser une action sur un objet dans un contexte donné.

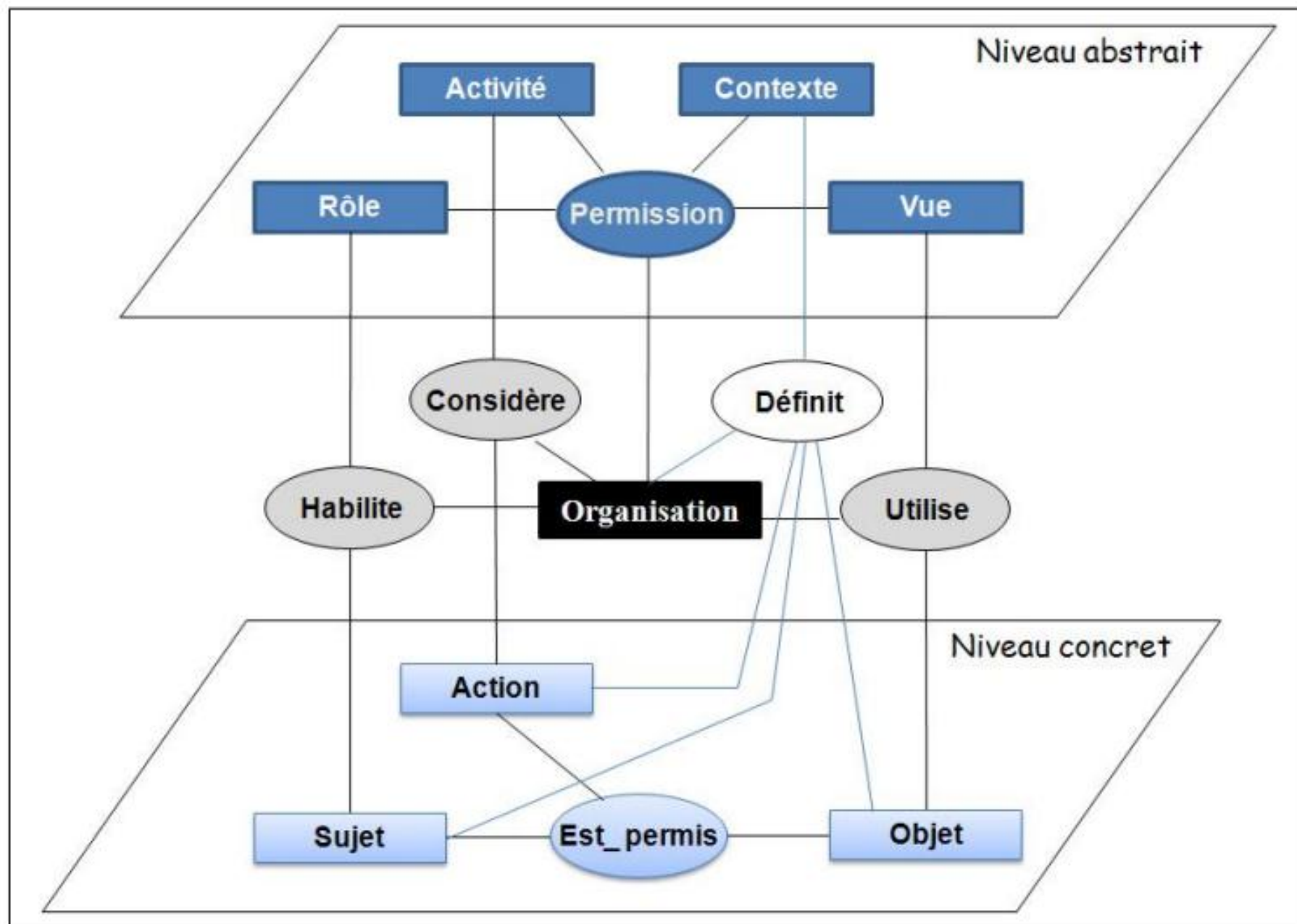
Est_Permis(s, o, ac) : signifie que le sujet `s` est autorisé à réaliser l'action `ac` sur l'objet `o`.

Est_Interdit(s, o, ac): signifie que le sujet `s` n'est pas autorisé à réaliser l'action `ac` sur l'objet `o`.

Est_Recommandé(s, o, ac): signifie que l'action `ac` sur l'objet `o` est recommandée mais non obligatoire pour le sujet `s`.

Est_Obligé(s, o, ac) signifie que l'action `ac` sur l'objet `o` est obligatoire pour le sujet `s`.

Par exemple, considérons un système de gestion de bibliothèque où les étudiants peuvent emprunter des livres. Si un étudiant "Alice" souhaite emprunter un livre "Introduction à l'intelligence artificielle", alors `Est_Permis(Alice, Livre IA, emprunter)` indiquerait qu'Alice est autorisée à emprunter ce livre. En revanche, si le même livre est réservé pour un cours spécifique, alors `Est_Interdit(Alice, Livre IA, emprunter)` pourrait être défini pour indiquer qu'Alice n'est pas autorisée à l'emprunter dans ce cas précis.



Le modèle de contrôle d'accès basé sur les attributs (ABAC)

Le modèle **ABAC (Attribute-Based Access Control)** est apparu avec le standard XACML (eXtensible Access Control Markup Language) dans le but d'assurer une meilleure flexibilité dans la gestion des accès.

Dans ce modèle :

Les propriétés des utilisateurs, des objets et des autres entités sont exprimées à l'aide d'attributs.

Un **attribut** est une propriété exprimée sous forme d'une paire (nom, valeur) et est associé à une entité dans le système.

Une **entité** peut être un sujet (utilisateur), une action, une ressource ou un environnement.

Le modèle **ABAC** permet de définir des politiques de contrôle d'accès en se basant sur ces attributs. Par exemple, une politique pourrait spécifier que seuls les utilisateurs ayant l'attribut "age" supérieur à 18 sont autorisés à accéder à certaines ressources sensibles. De même, une politique pourrait limiter l'accès à une action particulière uniquement aux utilisateurs ayant l'attribut "rôle" égal à "administrateur". En utilisant les attributs associés aux différentes entités du système, ABAC offre une flexibilité accrue dans la définition des politiques de contrôle d'accès.

Le modèle de contrôle d'accès basé sur les attributs (ABAC) (2)

Dans le contexte du contrôle d'accès, les entités sont regroupées en trois catégories principales : Sujet, Ressource et Action. Chacune de ces catégories possède des attributs qui définissent des caractéristiques spécifiques.

Par exemple :

Les attributs du Sujet peuvent inclure le Nom, le Département, l'Âge, le Rôle, le Bureau, etc.

Les attributs de l'Action peuvent comprendre l'Identification et le Type.

Les attributs de la Ressource peuvent inclure le Type et l'Identification.

Le modèle de contrôle d'accès basé sur les attributs (ABAC) (3)

Chaque attribut est associé à une valeur qui caractérise l'entité correspondante.

Par exemple :

Prénom(Sujet) = John

- Nom(Sujet) = Doe
- Service(Sujet) = Développement informatique
- Niveau d'accès(Sujet) = Administrateur
- Numéro d'identification(Sujet) = 12345
- Type(Action) = Lecture
- Date et heure(Action) = 2024-04-23 14:30
- Lieu(Action) = Bureau 102
- Nom de fichier(Ressource) = Rapport_final.docx
- Type de document(Ressource) = Document Word
- Propriétaire(Ressource) = Équipe de projet
- Date de création(Ressource) = 2024-04-15

Le modèle de contrôle d'accès basé sur les attributs (ABAC) (4)

Une requête de contrôle d'accès est un ensemble d'éléments, chaque élément étant un attribut suivi de sa valeur, qui spécifie les paramètres de la requête. Par exemple :

(Nom(Sujet) = Alice) et (Type(Action) = Écriture) et (Type(Ressource) = Document)

Cette requête indique que l'utilisateur nommé "Alice" souhaite effectuer l'action d'écriture sur une ressource de type document. Cela signifie que "Alice" veut écrire sur un document spécifique.

**(Département(Sujet) = Finance) et (Type(Action) = Consultation) et
(Propriétaire(Ressource) = Service Juridique)**

Cette requête indique qu'un utilisateur du département des finances souhaite consulter une ressource qui appartient au service juridique. Cela pourrait représenter un scénario où quelqu'un du département financier doit consulter des documents légaux détenus par le service juridique.

Le modèle de contrôle d'accès basé sur les attributs (ABAC) (5)

Les règles de contrôle d'accès définissent les conditions sous lesquelles un accès est autorisé ou refusé en fonction des attributs du sujet, de l'action et de la ressource (cibles). Elles sont exprimées sous forme d'expressions booléennes.

Permettre si :

L'accès est permis si le rôle du sujet est "Professeur" ou "Personnel".

L'action est "Emprunt-Livre".

Le type de la ressource n'est pas "Réserve".

Par exemple, un accès est autorisé si l'utilisateur est un professeur ou du personnel, et s'il essaie d'emprunter un livre qui n'est pas réservé.

Refuser si :

L'accès est refusé si le rôle du sujet est "Étudiant".

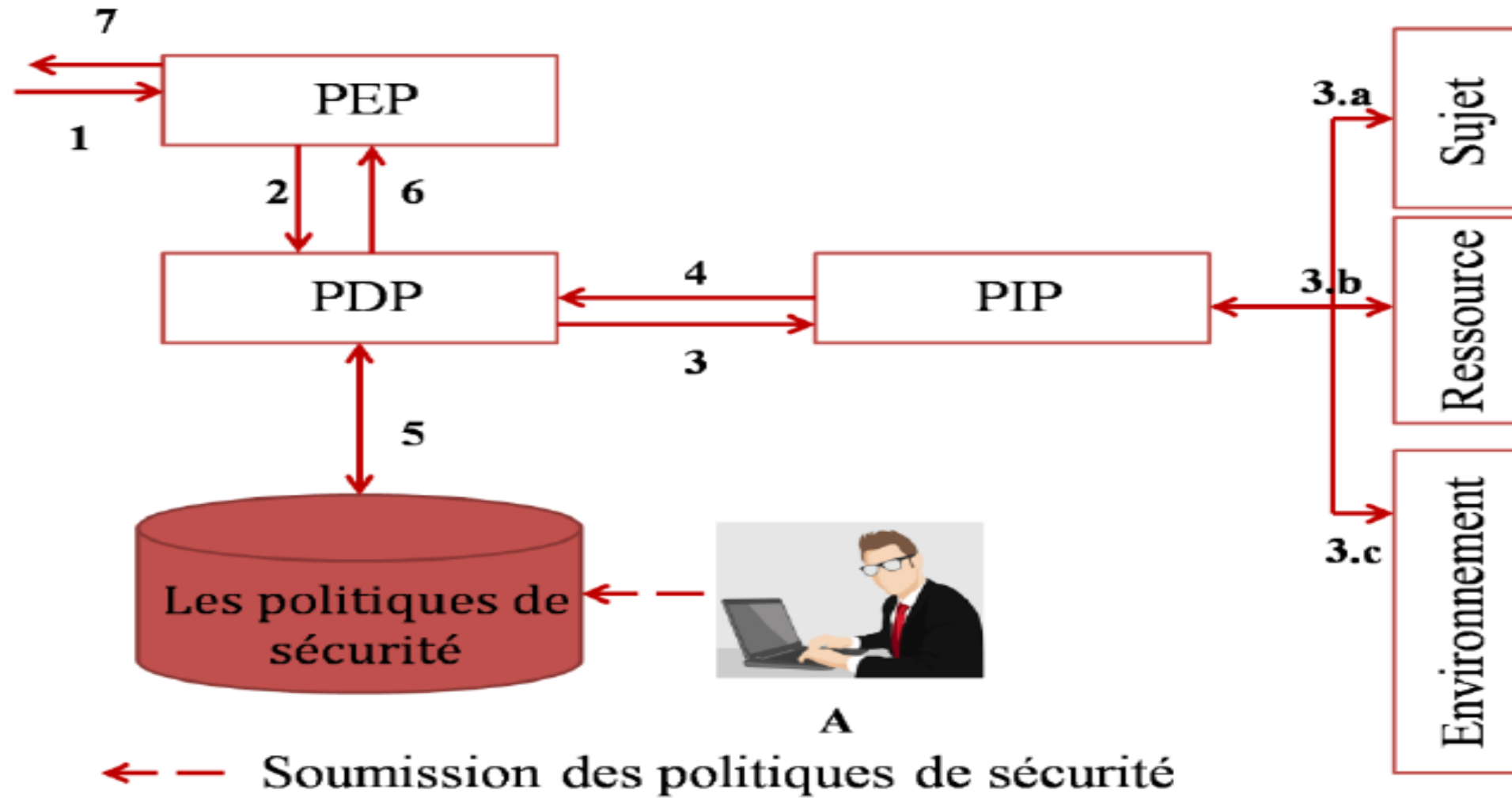
L'action est "Emprunt-Livre".

Le type de la ressource est soit "Revue" soit "Réserve".

Par exemple, l'accès est refusé si l'utilisateur est un étudiant et qu'il essaie d'emprunter une revue ou un livre réservé.

Ces règles définissent des conditions spécifiques qui doivent être satisfaites pour permettre ou refuser l'accès en fonction du rôle de l'utilisateur, de l'action demandée et du type de ressource.

Architecture ABAC



Architecture ABAC (2)

L'architecture ABAC (Attribute-Based Access Control) est un modèle de contrôle d'accès qui utilise des attributs pour déterminer l'autorisation d'accès aux ressources. Au lieu de se baser uniquement sur les rôles des utilisateurs, comme c'est le cas dans le modèle RBAC (Role-Based Access Control), l'ABAC prend en compte un large éventail d'attributs liés aux sujets (utilisateurs), aux ressources et à l'environnement. Ces attributs sont évalués par un moteur de décision pour déterminer si un accès doit être autorisé ou refusé. L'architecture ABAC comprend des composants tels que

Architecture ABAC (3)

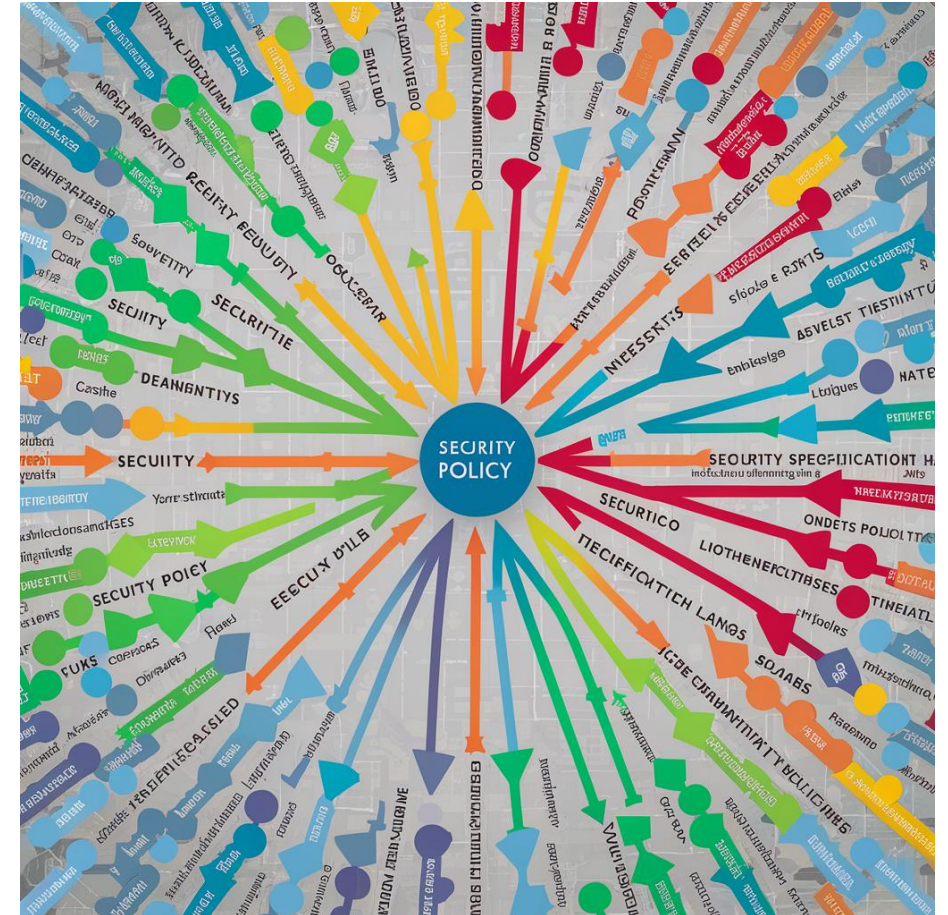
- **PEP** (Policy Enforcement Point) : est le point d'application de la politique, c'est ce composant qui se chargera de créer les requêtes d'accès et de les envoyer au PDP. En outre, dès la réception d'une réponse du PDP, le PEP la transmettra à l'utilisateur.
- **PDP** (Policy Decision Point) : est le point qui se chargera de prendre la décision d'accès en comparant les valeurs des attributs de la requête d'accès avec celles des attributs de la politique de sécurité.
- **PIP** (Policy Information Point) : est le point d'information de la politique, ce point se chargera de récupérer les attributs manquants concernant les ressources, les utilisateurs et l'environnement afin que le PDP puisse évaluer la politique de sécurité.
- **La base des politiques de sécurité** : cette base contient toutes les politiques du système rédigées utilisant le langage XACML qui est basé sur XML.

Traitement d'une requête de contrôle d'accès

Après la soumission des politiques de sécurité rédigées en XACML par l'administrateur du système (flèche discontinue dans Diapo 89), à chaque nouvelle demande de contrôle d'accès reçue par le PEP (1). Une requête XACML (Diapo 89) est créée et envoyée au PDP (2). Dès la réception de cette requête, le PDP demande au PIP les valeurs manquantes des attributs (3) tels que les attributs concernant l'utilisateur (3.a), les attributs concernant les ressources (3.b) ou ceux concernant l'environnement (3.c). Après la réception de ces attributs, (4) le PDP évalue la requête en comparant les valeurs des attributs contenues dans la requête avec celles des attributs de la politique de sécurité applicable qui est stockée dans la base des politiques de sécurité (5). Ensuite, le PDP envoie une réponse XACML contenant le résultat de l'évaluation (6). Dès la réception de ce résultat par le PEP, il transmet la réponse à l'utilisateur (7).

Les langages de spécification de politiques de sécurité

Les langages de spécification de politiques de sécurité sont des outils utilisés pour définir et formaliser les règles et les politiques qui régissent le contrôle d'accès aux ressources d'un système informatique. Ces langages fournissent un moyen de décrire de manière précise et formelle les autorisations et les restrictions concernant l'accès aux données, aux fonctionnalités et aux services.



Les caractéristiques d'un langage de spécification des politiques



Les caractéristiques d'un langage de spécification des politiques (2)

Expressivité : Un langage de spécification de politique de sécurité doit être expressif, ce qui signifie qu'il doit permettre de définir des politiques de sécurité complexes et variées. Cela inclut la capacité à exprimer des règles de contrôle d'accès avec des conditions, des combinaisons logiques et des autorisations granulaires pour différents utilisateurs, ressources et actions.

Clair et lisible: La clarté et la lisibilité du langage sont essentielles pour faciliter la compréhension des politiques de sécurité par les administrateurs, les auditeurs et les autres parties prenantes. Un langage bien conçu doit utiliser une syntaxe cohérente, des termes compréhensibles et une structure logique qui rendent les politiques faciles à interpréter et à vérifier.

Extensibilité: La capacité d'un langage à être étendu est importante car elle permet d'ajouter de nouvelles fonctionnalités, de nouvelles règles et de nouveaux mécanismes de contrôle au fur et à mesure que les besoins en matière de sécurité évoluent. Un langage extensible peut s'adapter aux changements dans les environnements informatiques et aux nouvelles menaces de sécurité sans nécessiter de modifications majeures de son architecture ou de sa syntaxe.

Les caractéristiques d'un langage de spécification des politiques (3)

Processus décisionnels efficaces: Un langage de spécification de politique de sécurité doit permettre une prise de décision efficace sur les autorisations d'accès aux ressources. Cela implique des mécanismes de traitement des requêtes d'accès qui sont rapides, précis et optimisés pour gérer un grand nombre de transactions simultanées tout en maintenant des performances élevées du système.

Sémantique succincte et non ambiguë: La sémantique d'un langage fait référence à la signification des termes et des expressions utilisés pour définir les politiques de sécurité. Une sémantique succincte et non ambiguë garantit que les politiques sont interprétées de manière cohérente et correcte par tous les composants du système. Cela réduit les risques d'interprétations erronées ou contradictoires des politiques, ce qui pourrait compromettre la sécurité du système.

XACML (eXtensibleAccess Control Markup Language)

XACML (eXtensible Access Control Markup Language) est un langage basé sur XML spécialement conçu pour la gestion du contrôle d'accès. Normalisé par OASIS en 2005, il fournit un cadre standardisé pour exprimer et gérer les politiques d'autorisation dans les systèmes informatiques.

XACML se compose de deux langages principaux :

1.Langage de politique d'autorisation: Ce langage permet de définir les politiques d'autorisation qui spécifient les règles et les conditions sous lesquelles un accès à une ressource est autorisé ou refusé. Ces politiques peuvent être exprimées de manière structurée à l'aide de balises XML, ce qui facilite leur création, leur gestion et leur interprétation par les systèmes informatiques.

2.Langage de requête et de réponse: Ce langage est utilisé pour formuler des requêtes d'autorisation et pour interpréter les réponses renvoyées par les entités responsables de la prise de décision sur les politiques (PDP - Policy Decision Point). Les requêtes d'autorisation peuvent spécifier les détails de l'accès demandé, tels que l'identité de l'utilisateur, la ressource ciblée et le type d'action demandé.

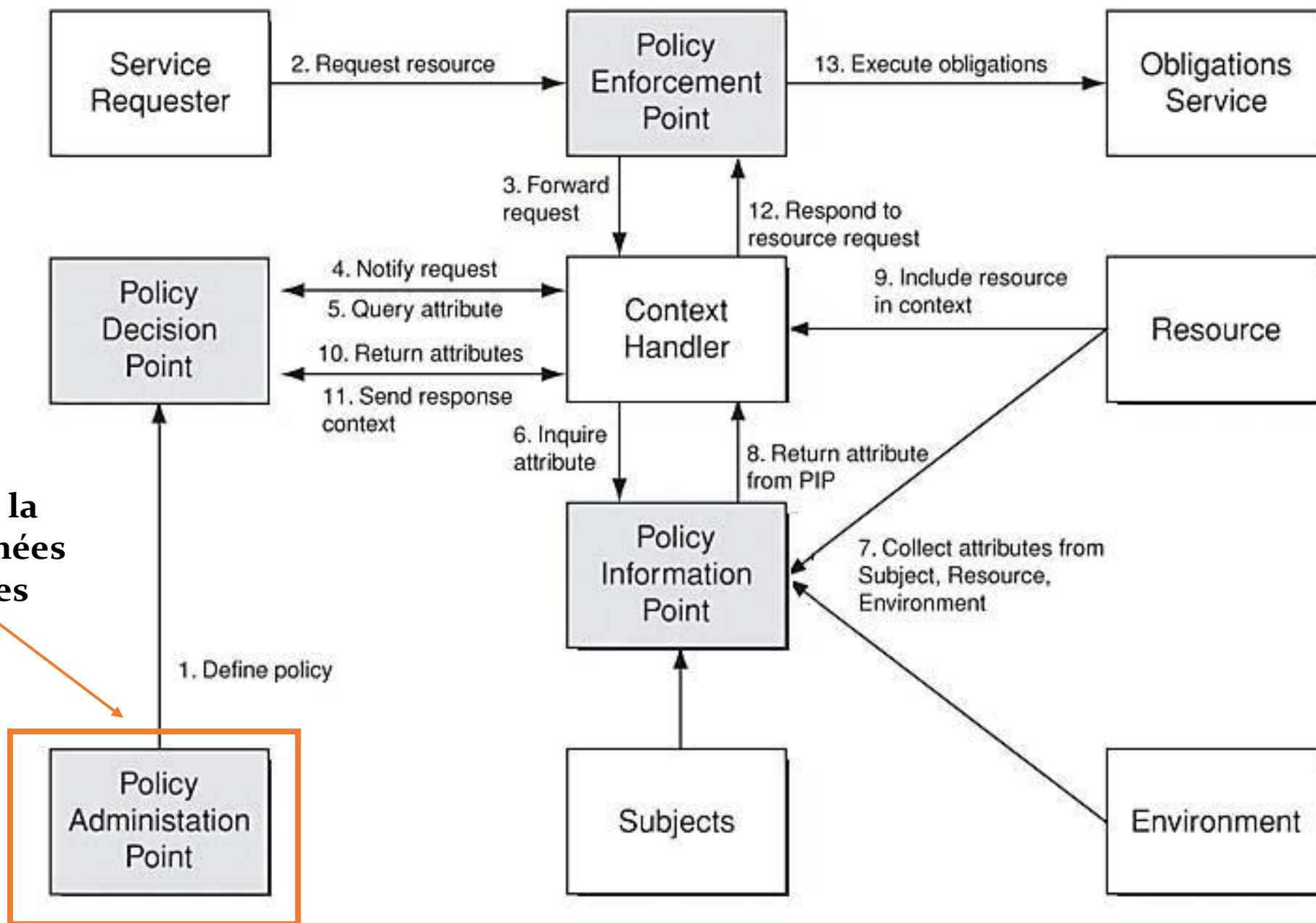
XACML (eXtensibleAccess Control Markup Language) (2)

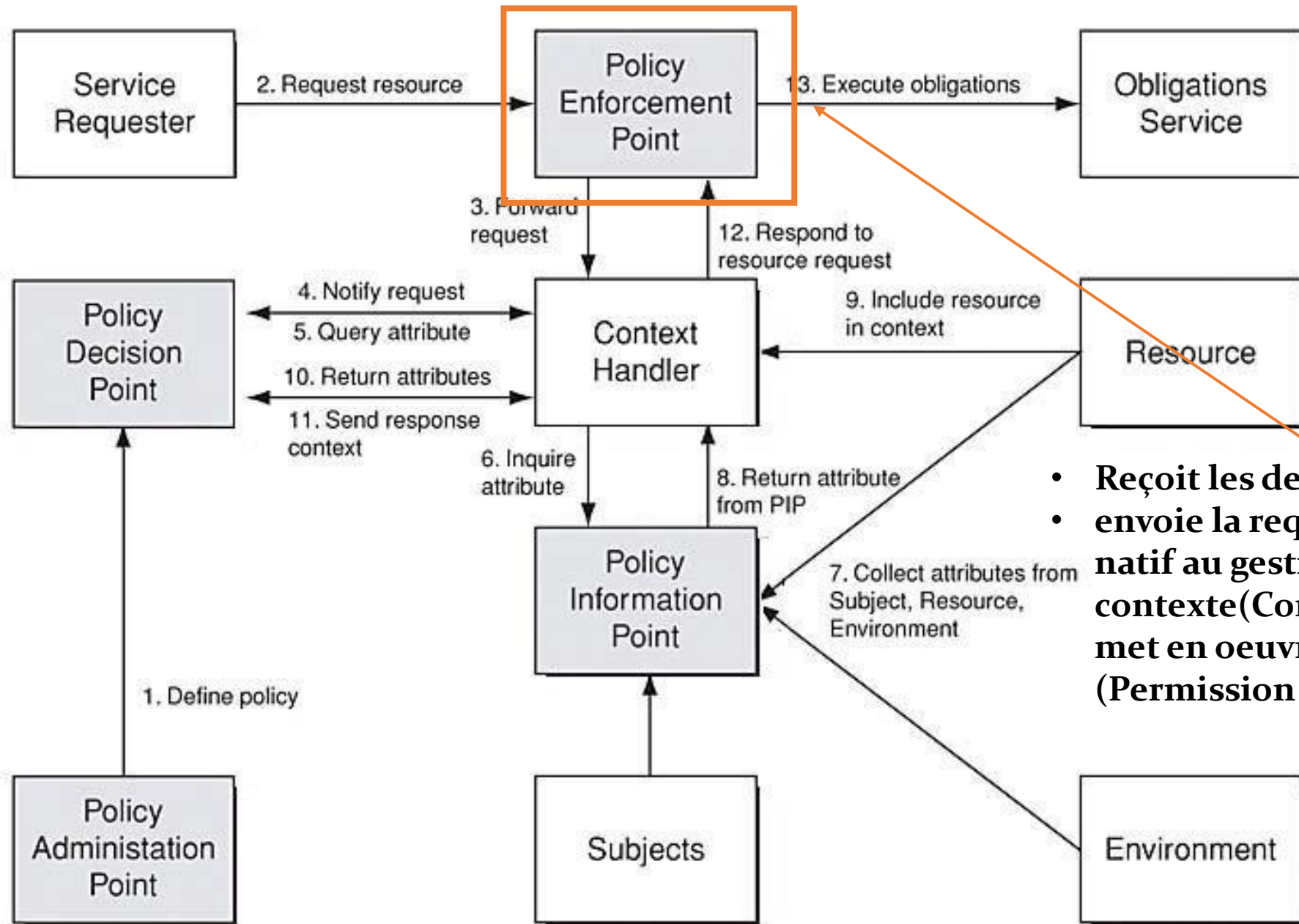
Les principales entités de l'architecture XACML comprennent :

Le point de décision de politiques (PDP - Policy Decision Point): Il s'agit de l'entité chargée de déterminer quelles politiques sont applicables à une requête d'accès spécifique et de prendre une décision d'autorisation en fonction de ces politiques. Le PDP évalue les requêtes d'accès par rapport aux politiques définies et renvoie une décision d'autorisation (accorder ou refuser) en conséquence. Cette entité joue un rôle central dans le processus de contrôle d'accès.

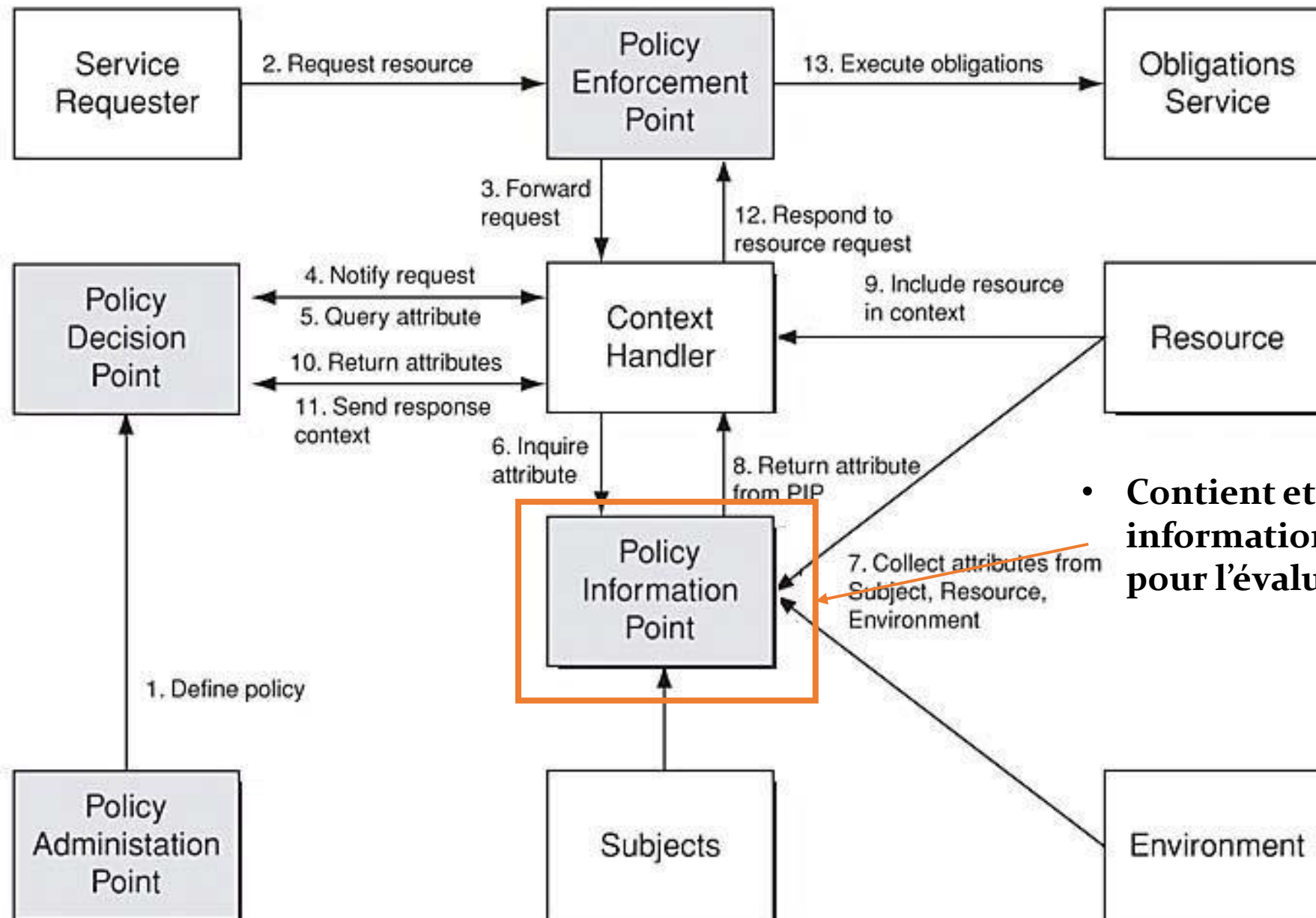
En résumé, XACML fournit un cadre flexible et extensible pour la gestion des politiques d'autorisation dans les environnements informatiques, permettant aux organisations de définir, de gérer et de mettre en œuvre des stratégies de contrôle d'accès conformes à leurs besoins spécifiques.

**Entretien de la
base de données
des politiques**

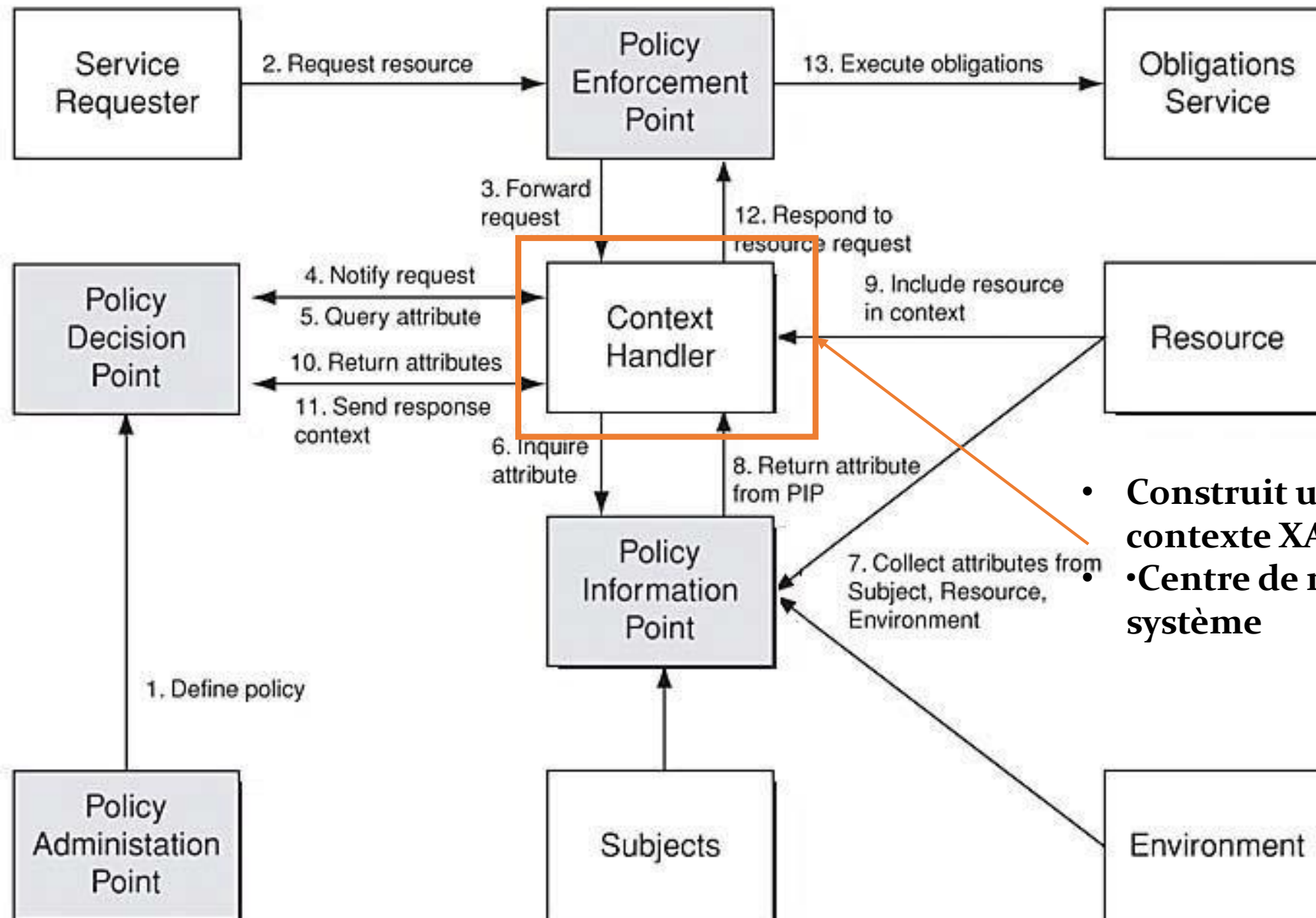




- Reçoit les demandes d'accès
- envoie la requête au format natif au gestionnaire de contexte (ContextHandler) et met en oeuvre les réponses (Permission ou refus).



- **Contient et Distribue les informations nécessaires pour l'évaluation des règles.**



- **Construit une demande contexte XACML.**
- **Centre de relai pour le système**

Traitement d'une requête d'accès XACML

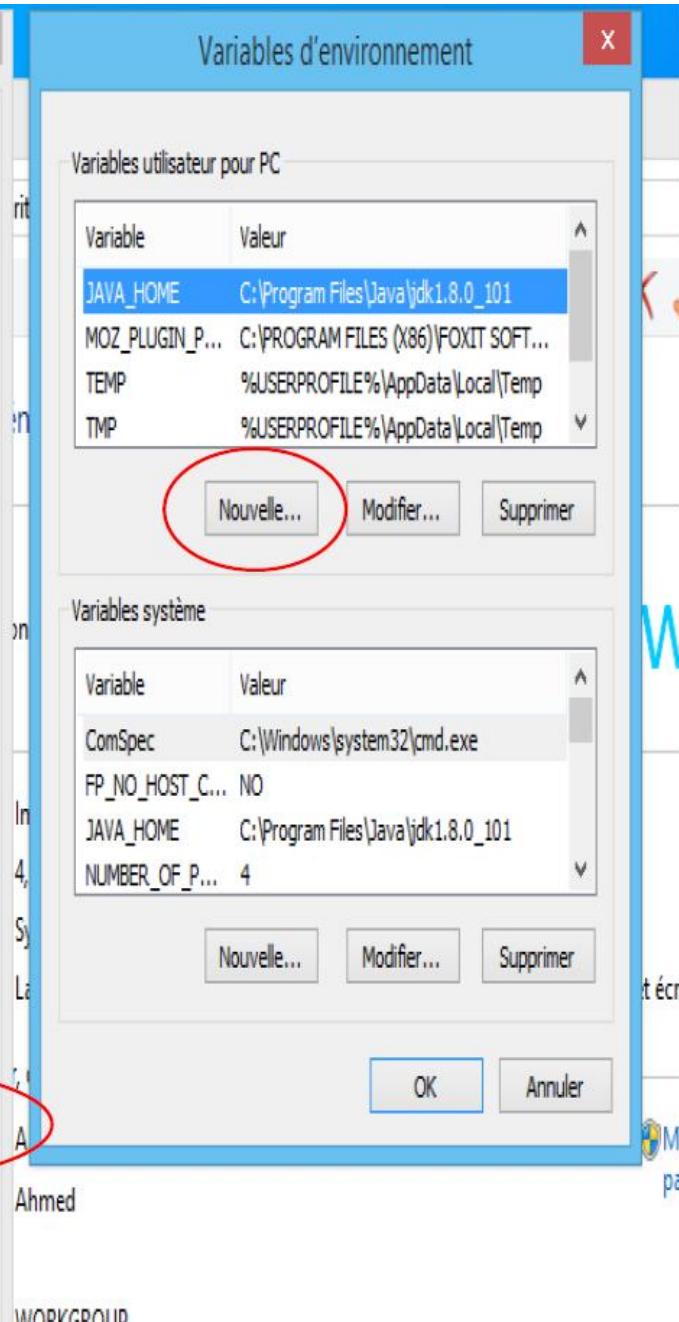
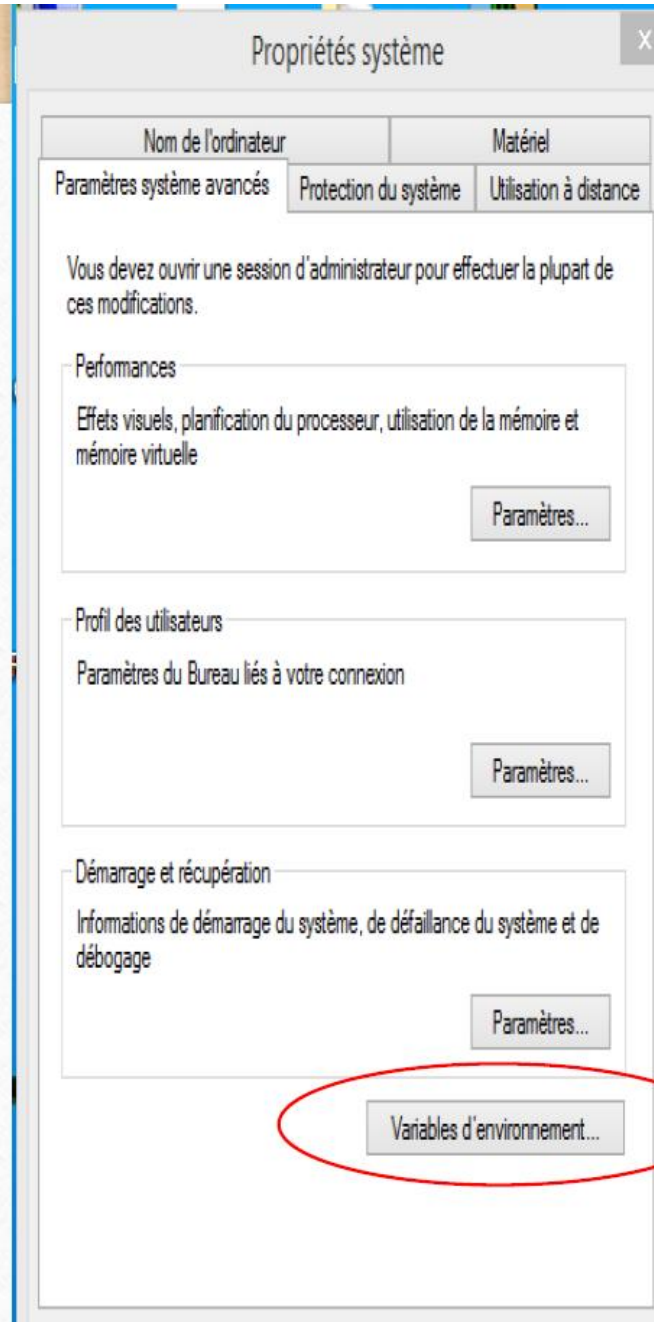
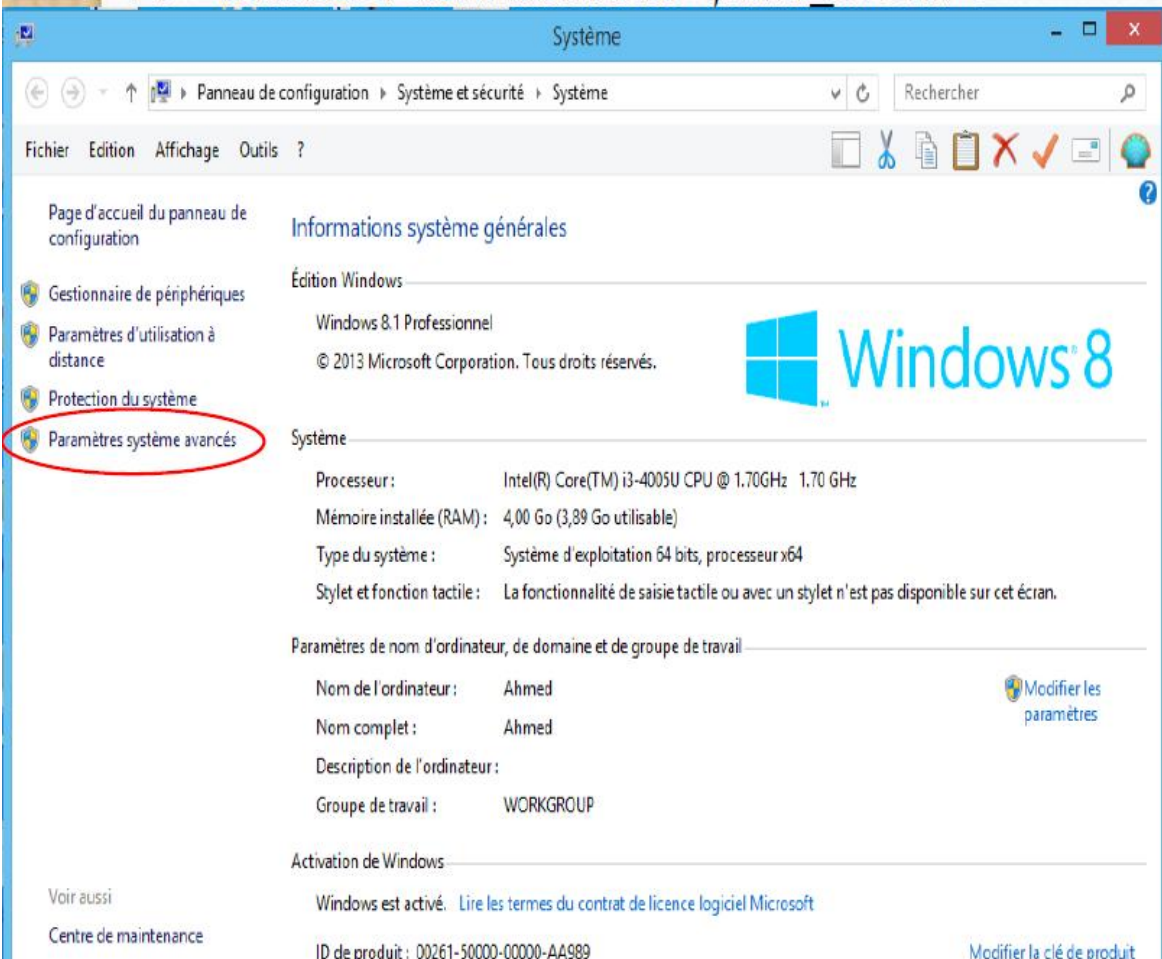
1. **Définition des politiques par le PAP** : Le Point d'Administration de Politiques (PAP) définit les politiques de sécurité ou l'ensemble de politiques disponibles pour le contrôle d'accès et les rend disponibles au Point de Prise de Décision (PDP).
2. **Demande d'accès** : Le demandeur d'accès, également appelé Access Requester, envoie une requête d'accès au Point de Prise en Charge de l'Exécution (PEP).
3. **Transmission de la requête au Context Handler** : Le PEP envoie la requête au format natif au gestionnaire de contexte (Context Handler).
4. **Traduction de la requête** : Le gestionnaire de contexte traduit la requête au format standard XACML et l'envoie au PDP.
5. **Demande d'attributs supplémentaires** : Le PDP peut demander au gestionnaire de contexte des attributs supplémentaires pour prendre une décision d'autorisation.
6. **Obtention des attributs** : Le gestionnaire de contexte demande à sa source d'attributs, également appelée PIP (Point d'Information sur les Attributs), les attributs demandés.

Traitement d'une requête d'accès XACML (2)

- 7. Récupération des attributs** : Le PIP obtient les attributs demandés.
- 8. Retour des attributs** : Le PIP retourne les attributs au gestionnaire de contexte.
- 9. Transmission des attributs au PDP** : Le gestionnaire de contexte envoie les attributs au PDP qui évalue la politique
- 10. Retour de la décision** : Le PDP retourne la décision d'autorisation au format standard au gestionnaire de contexte.
- 11. Traduction de la réponse** : Le gestionnaire de contexte traduit la réponse au format natif utilisé par le PEP et l'envoie à ce dernier.
- 12. Exécution des obligations** : Le PEP remplit les obligations et, en se fondant sur la décision d'autorisation adressée par le PDP, soit permet ou interdit l'accès.

Les étapes d'installation de WSO2

1. JDK 1.8.0 C:\Program Files\Java\jdk1.8.0_101
2. Variable d'environnement JAVA_HOME



Il faut installer jdk 17

set JAVA_HOME=C:\chemin\vers\votre\JDK

set PATH=%PATH%;%JAVA_HOME%\bin

CD C:\Users\Asus\Desktop\wso2is-7.0.0\wso2is-7.0.0\bin mettez le chemin de votre WSO2 puis exécutez la commande: **wso2server.bat**

A screenshot of a Windows Command Prompt window. The title bar at the top reads 'C:\ Invite de commandes'. The window has standard Windows window controls (minimize, maximize, close) on the right. The command prompt shows the following text: 'Microsoft Windows [version 6.1.7600] Copyright (c) 2009 Microsoft Corporation. Tous droits réservés.' followed by the command 'C:\Users\user>cd Desktop' and the resulting prompt 'C:\Users\user\Desktop>'.

```
C:\ Invite de commandes

Microsoft Windows [version 6.1.7600]
Copyright (c) 2009 Microsoft Corporation. Tous droits réservés.

C:\Users\user>cd Desktop
C:\Users\user\Desktop>
```

```
[2017-04-28 18:24:51,003] INFO {org.jaggeryjs.jaggery.app.mgt.TomcatJaggeryWebappsDeployer} - Deployed webapp: StandardEngine[Catalina].StandardHost[localhost].StandardContext[/portal].File[C:\Users\PC\Desktop\WSO2IS~1.0\WSO2IS~1.0\bin\..\repository\deployment\server\jaggeryapps\portal]
[2017-04-28 18:24:51,024] INFO {org.apache.tomcat.util.net.NioSelectorPool} - Using a shared selector for servlet write/read
[2017-04-28 18:24:51,120] INFO {org.apache.tomcat.util.net.NioSelectorPool} - Using a shared selector for servlet write/read
[2017-04-28 18:24:51,230] INFO {org.wso2.carbon.core.init.JMXServerManager} - JMX Service URL : service:jmx:rmi://localhost:11111/jndi/rmi://localhost:9999/jmxrmi
[2017-04-28 18:24:51,233] INFO {org.wso2.carbon.core.internal.StartupFinalizerServiceComponent} - Server : WSO2 Identity Server-5.3.0
[2017-04-28 18:24:51,235] INFO {org.wso2.carbon.core.internal.StartupFinalizerServiceComponent} - WSO2 Carbon started in 156 sec
[2017-04-28 18:24:53,543] INFO {org.wso2.carbon.ui.internal.CarbonUIServiceComponent} - Mgt Console URL : https://localhost:9443/carbon/
[2017-04-28 18:24:53,617] INFO {org.wso2.carbon.identity.authenticator.x509Certificate.internal.X509CertificateServiceComponent} - X509 Certificate Servlet activated successfully..
[2017-04-28 18:25:33,879] INFO {org.wso2.carbon.core.services.util.CarbonAuthenticationUtil} - 'admin@carbon.super [-1234]' logged in at [2017-04-28 18:25:33,325+0100]
```

Identity & Access Manag xWSO2 Management Con x

Non sécurisé | https://127.0.0.1:9443/carbon/admin/index.jsp?loginStatus=true

☆🔍🔒⋮

WSO2 Identity Server

Management Console

Signed-in as: admin@carbon.super | Sign-out | Docs | About

Home

Identity

Users and Roles

Add

List

User Stores

Add

List

Claims

Add

List

Service Providers

Add

List

Resident

Identity Providers

Add

List

Resident

Entitlement

PAP

Policy Administration

WSO2 Identity Server Home

Welcome to the WSO2 Identity Server Management Console

Server

Host	localhost
Server URL	local://services/
Server Start Time	2017-04-28 18:22:14
System Up Time	0 day(s) 0 hr(s) 5 min(s) 9 sec(s)
Version	5.3.0
Repository Location	file:/C:/Users/PC/Desktop/WSO2IS~1.0/WSO2IS~1.0/bin/./repository/deployment/server/

Operating System

OS Name	Windows 8.1
OS Version	6.3

Operating System User

Country	FR
Home	C:\Users\PC
Name	PC

Policy Administration

 [Add New Entitlement Policy](#)

Policy Type ALL Search Policy

Select all in this page | Select none  Delete  Publish  Publish All

Available Entitlement Policies	
☐ TestPolicy	 Policy  Edit  Versions  Publish To My PDP  Try  View Status

Add New Policy

Policy creation methods	
 Simple Policy Editor	You can define simple access control rules using this editor. Then you can covert these rules in to XACML 3.0 policy.
 Basic Policy Editor	You can create a basic XACML 3.0 policy. Categories are limited to Resource, Action, Subject and Environment. This editor is configurable. You can do it from here
 Standard Policy Editor	You can create a normal XACML 3.0 policy. Here you can define custom categories, attributels and DataTypes. Also you can add Obligations and Advices in to your rules and policy. This editor is configurable. You can do it from here
 Policy Set Editor	You can create a XACML 3.0 policy sets. Here you can define Policy Set Target, Obligations, Advices and References to already defined policies or policy sets. This editor is configurable. You can do it from here
 Import Existing Policy	You can import existing XACML policy from file system or from carbon registry

Create XACML Policy

Entitlement Policy Name*

Entitlement Policy Description

This policy is based on

Resource 

Resource which is equals to

Child resource

User

Action

Environment

Age 

Time 



Create XACML Policy

Entitlement Policy Name*	policy-one												
Entitlement Policy Description	accès a la ressource R1												
This policy is based on	Resource												
Resource which is equals to	R1												
<table><thead><tr><th>Child resource</th><th>User</th><th>Action</th><th>Environment</th></tr></thead><tbody><tr><td> </td><td> Role </td><td> ETUDIANT </td><td> READ </td></tr><tr><td></td><td> Domain </td><td> DOMAIN ONE </td><td> </td></tr></tbody></table>		Child resource	User	Action	Environment		Role	ETUDIANT	READ		Domain	DOMAIN ONE	
Child resource	User	Action	Environment										
	Role	ETUDIANT	READ										
	Domain	DOMAIN ONE											
Finish Cancel													

Edit XACML Policy

Entitlement Policy Name*	policy-one												
Entitlement Policy Description													
This policy is based on	Resource												
Resource which is equals to	R1												
<table><thead><tr><th>Child resource</th><th>User</th><th>Action</th><th>Environment</th></tr></thead><tbody><tr><td> </td><td> Role </td><td> ETUDIANT </td><td> READ </td></tr><tr><td></td><td> Domain </td><td> DOMAIN ONE </td><td> </td></tr></tbody></table>		Child resource	User	Action	Environment		Role	ETUDIANT	READ		Domain	DOMAIN ONE	
Child resource	User	Action	Environment										
	Role	ETUDIANT	READ										
	Domain	DOMAIN ONE											
<table><thead><tr><th>Child resource</th><th>User</th><th>Action</th><th>Environment</th></tr></thead><tbody><tr><td> </td><td> Role </td><td> ENSEIGNANT </td><td> WRITE </td></tr><tr><td></td><td> Domain </td><td> DOMAIN ONE </td><td> </td></tr></tbody></table>		Child resource	User	Action	Environment		Role	ENSEIGNANT	WRITE		Domain	DOMAIN ONE	
Child resource	User	Action	Environment										
	Role	ENSEIGNANT	WRITE										
	Domain	DOMAIN ONE											
Finish Cancel													

	F-notes	F-cours	F-administrative
Etudiant	LIRE	LIRE	LIRE
Enseignant	LIRE /ECRIRE	LIRE/ECRIRE	LIRE
Chef de département	LIRE	LIRE	LIRE/ECRIRE
Agent administrative	LIRE	LIRE	LIRE/ECRIRE

Référence

D. E. Bell et L. J. LaPadula. Secure computer systems: Unified exposition and multics interpretation. Technical Report ESD-TR-73-306, The MITRE Corporation, Mars 1976.

Eric Yuan, Jin Tong. Attribute Based Access Control, A New Access Control Approach for Service Oriented Architectures (SOA), New Challenges for Access Control Workshop, Ottawa, ON, Canada, 2005

Bassem Nasser. Organisation Virtuelle : Gestion de politiques de contrôle d'accès inter domaine, Thèse de doctorat, Université Paul Sabatier, Toulouse, France, novembre 2006, 234 pp

M.A. Harrison, W.L. Ruzzo, and J.D. Ullman: Protection in Operating Systems. Communications of the ACM, August 1976.

Marwa El Houri, Un modèle formel pour exprimer des politiques dynamiques pour contrôle d'accès et négociation dans un environnement distribué (2010)

Romuald THION, structuration relationnelle des politiques de contrôle d'accès (2008)

M. ABDELJEBAR AMEZIANE EL HASSANI, Le contrôle d'accès des réseaux et grandes infrastructures critiques distribuées (2016)

